

Short summary/Objective: Developing advanced machine learning systems for temporal radar data prediction and analysis by: (i) implementing distributed training infrastructure for large-scale neural networks; (ii) building temporal prediction models using masked auto-encoding architectures; (iii) applying classical signal processing techniques for radar blob detection and tracking.

Summary: Establishing foundational methodologies for processing high-resolution radar data through distributed training strategies that efficiently scale models across multiple GPU systems; developing multi-frame prediction capabilities using masked auto-encoding with variable context mechanisms for temporal radar sequences; and implementing classical algorithmic approaches for maritime surveillance and object detection, with current work addressing limitations in detecting small distant objects and demonstrating feasibility of deep learning and traditional signal processing fusion for radar-based applications.

Sea-ing in Binary: Distributed Deep Learning Infrastructure in Spatio-Temporal Space

Part 1: Overview of Outcomes

Extracting predictive insight from high-resolution temporal radar data demands reconciling sparse, noisy returns with the computational scale required for multi-frame forecasting and small-target detection. Over the reporting interval, three integrated efforts addressed this challenge.

(1) A temporal masked auto-encoding architecture was adapted from Video MAE V1 to handle radar-specific spatial structures, successfully generating one-to-two frame predictions from single inputs and establishing a variable-context mechanism that scales performance across one to two-hundred-fifty-five historical frames, with quantitative gains plateauing beyond twenty frames as sea-clutter chaos dominates.

(2) Full-resolution processing at the native 1735px scale was pursued through architectural modifications that increase attention complexity by a factor of eight relative to downsampled inputs, necessitating memory-efficient implementations to preserve fine-grained signatures of distant buoys and vessels.

(3) Four distributed training strategies (data, model, pipeline, and tensor parallelism) were systematically evaluated to accommodate thirteen-billion-parameter models that exceed single-GPU capacity, with sharding implementations balancing computation across devices while minimizing inter-node communication overhead.

Part 2: Temporal Masked Auto-Encoding for Radar Frame Prediction

Radar data presents unique challenges for temporal prediction tasks due to its sparse, noisy nature and the complex physical phenomena it captures. Unlike video data where adjacent frames typically exhibit high correlation through smooth motion, radar returns can change dramatically between pulses due to sea clutter, weather effects, and rapidly maneuvering targets. This section describes the development of temporal masked auto-encoding approaches specifically designed to handle these characteristics of radar data.

Part 2b: Model Development and Architecture

A temporal masked auto-encoding approach has been implemented for radar frame prediction tasks. This architecture builds upon the masked auto-encoder (MAE) paradigm originally developed for image processing and feature extraction, adapting it to handle temporal sequences of radar data. The fundamental principle involves masking portions of input radar frames and training the model to reconstruct these masked regions based on visible context, encouraging the network to learn meaningful representations of radar signatures and temporal dynamics rather than simply memorizing training data.

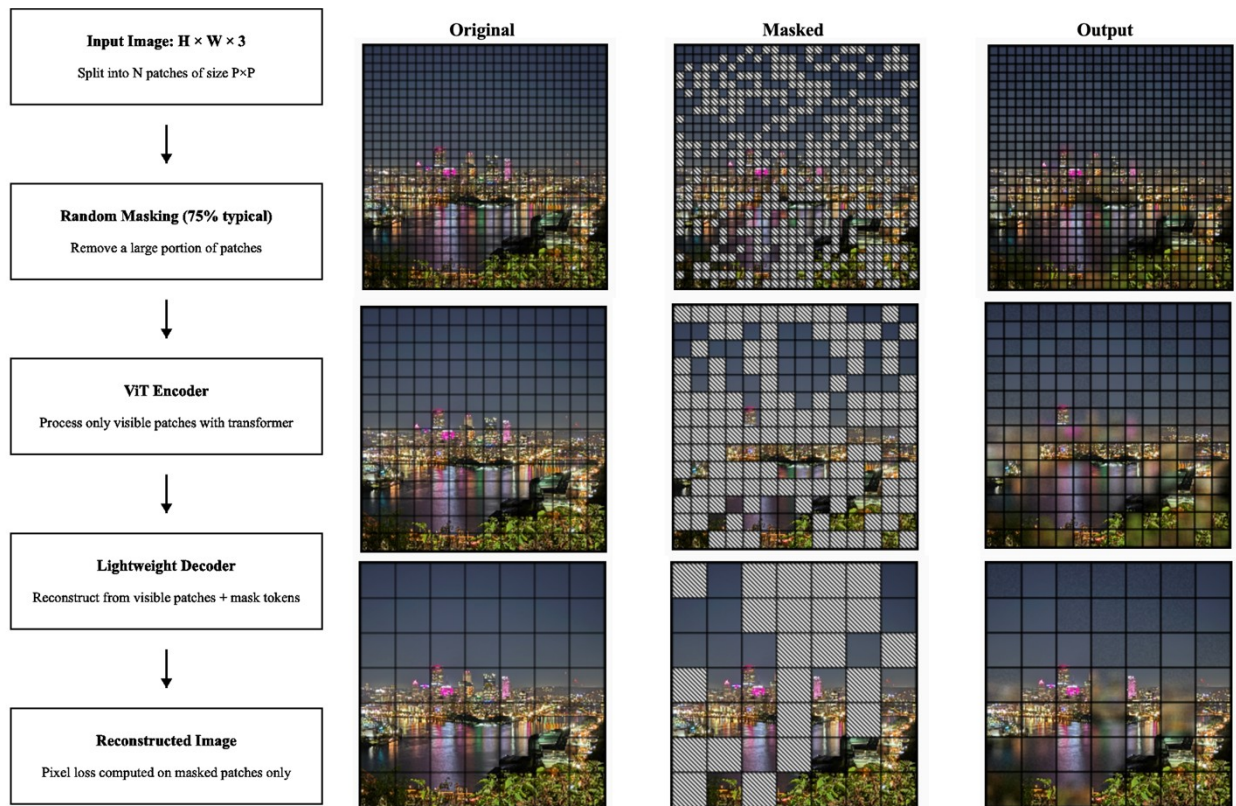


Figure 1: Masked Auto Encoder Architectural Diagram With Varied Patch Size Examples

Initial experimental results demonstrate reasonable performance in frame prediction capabilities. The model successfully generates one to two subsequent radar frames given a single initial input frame. This predictive capability suggests that the model has learned to capture essential temporal patterns in radar data, including the evolution of sea clutter, persistent land features,

and some aspects of target motion. The prediction horizon of 1-2 frames corresponds to approximately 1-2 seconds in real time for 60RPM, or hi-speed, radar pulse repetition frequencies, providing a meaningful lookahead window for tracking and prediction applications.

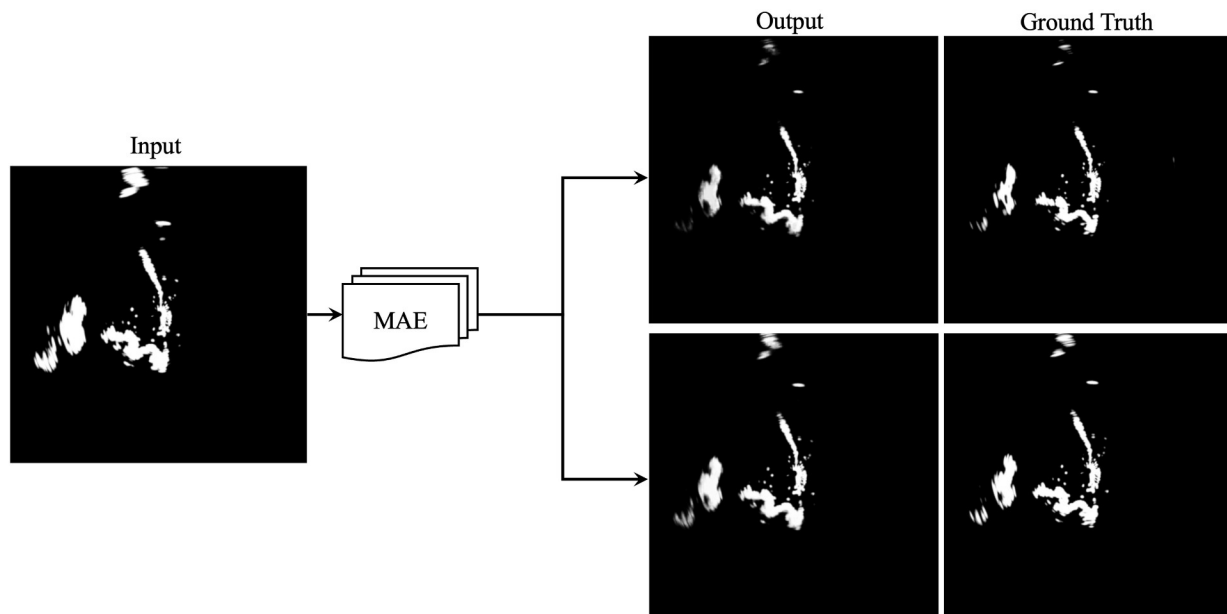


Figure 2: Temporal-MAE Predicting Future Frames – Compared To GT Data

The model architecture employs a vision transformer backbone rather than convolutional networks. This design choice is motivated by several factors, primarily the transformers' ability to capture long-range dependencies across the radar image and their superior scalability to high-resolution inputs through patch-based processing. The self-attention mechanism in transformers allows the model to dynamically weigh the importance of different spatial regions when making predictions, which is particularly valuable for radar data where targets may appear at arbitrary locations.

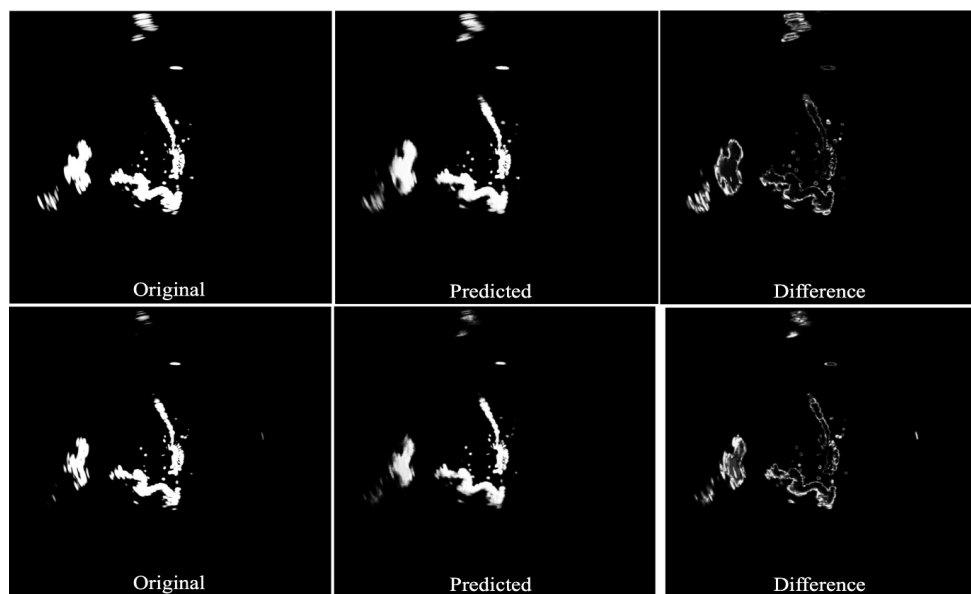


Figure 3: Frame 1 and 2 compared to original, along with Diff map

Part 2b: Variable Context Mechanism for Extended Temporal Modeling

Future development plans include extension to multi-pulse length models capable of handling variable-length temporal sequences. Traditional fixed-context models require a predetermined number of input frames, limiting their flexibility in real-world deployment scenarios where available history may vary. A variable context mechanism has been developed to address this limitation, enabling the incorporation of additional historical frames as radar operation duration increases. This adaptive approach allows the model to leverage whatever temporal information is available, from minimal context in cold-start scenarios to extensive history during sustained operations.

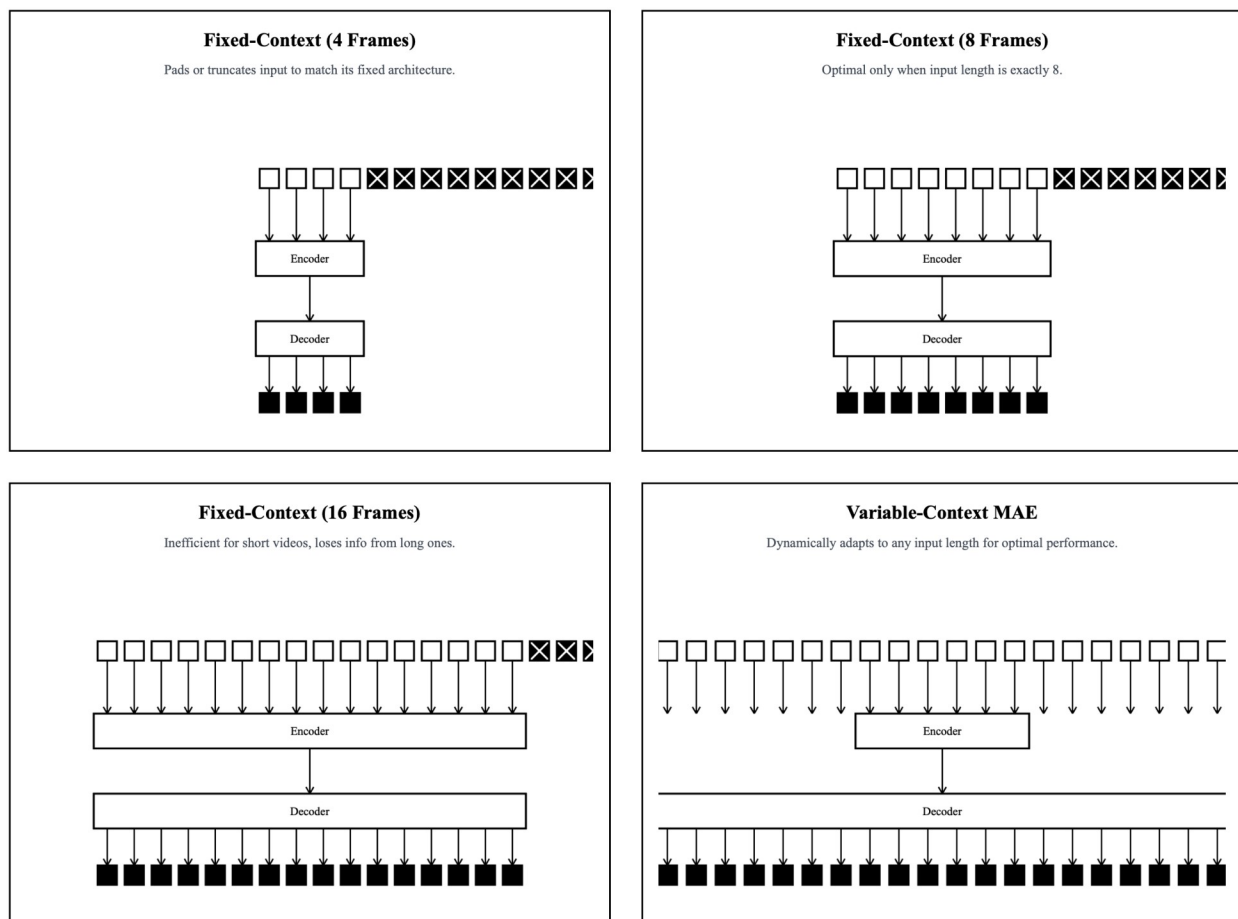


Figure 4: Fixed Context Temporal MAE v. Dynamic Context

The variable context mechanism operates by maintaining a dynamic memory buffer that accumulates radar frames over time. As new frames arrive, they are incorporated into the model's context window up to a maximum capacity determined by computational constraints. This approach may be categorized under long-context designs, a growing area of research in sequence modeling. The key innovation lies in the model's ability to effectively utilize contexts of varying lengths without requiring separate training or architecture modifications for each context size. This is achieved through learned positional embeddings that can accommodate arbitrary sequence lengths and attention mechanisms that naturally scale to different input sizes.

Performance analysis across varying context lengths demonstrates clear improvement trends as more historical frames become available. Quantitative evaluation using frame reconstruction metrics (mean squared error, peak signal-to-noise ratio) shows consistent gains from 1 frame up to approximately 255 frames of context. Beyond this point, returns diminish as computation time for training fundamentally increases and takes too long to train, i.e. training was not completed before running model. The performance improvement is non-linear, with steep gains in the first 10-20 frames (corresponding to 10-20 seconds of history) followed by more gradual improvements as additional context is added.

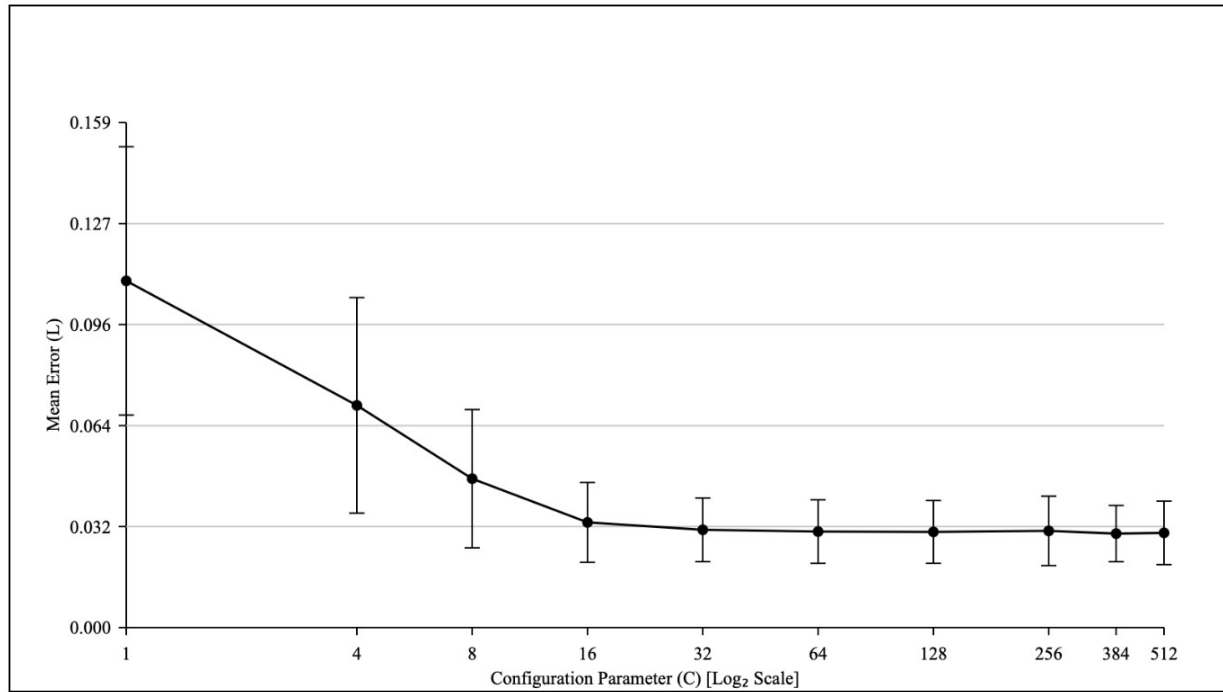


Figure 5: MSE v. Number of Frames in Context

The validation demonstrates that the variable context approach successfully captures temporal patterns at multiple timescales. Short-term context (1-10 frames) proves essential for predicting immediate frame-to-frame evolution and fast-moving targets. Medium-term context (10-50 frames) captures recurring patterns such as sea state variations and periodic scanning artifacts. Long-term context (50+ frames) helps establish stable baselines for land features and slowly evolving environmental conditions. This multi-scale temporal modeling capability distinguishes the variable context approach from simpler fixed-window methods.

Part 3: Model Scaling Challenges

Native radar data is captured at approximately 868 range bins and 800 pulses, then when converting into a cartesian image, the result is a 1735×1735 pixel resolution, representing the full fidelity of the sensing system. However, processing this high-resolution data presents significant computational challenges. Previous experiments operated at downsampled 224×224 resolution to fit within available GPU memory constraints. Current efforts focus on integrating full resolution

capabilities into the model architecture, enabling the preservation of fine-grained spatial details critical for detecting small distant targets such as buoys and boats.

The underlying architectural design is built upon the Video MAE V1 framework, originally developed for video understanding tasks. This architecture has been adapted for radar data processing through several key modifications: specialized patch embedding strategies that account for radar-specific spatial structures, modified attention mechanisms that can handle the increased token count from higher resolutions, and customized normalization schemes appropriate for radar intensity distributions. The Video MAE V1 foundation provides a robust starting point, but substantial engineering is required to scale it to full-resolution radar processing.

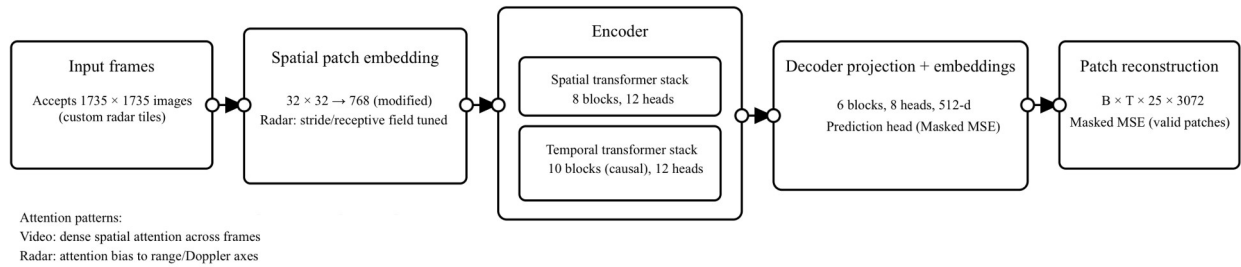


Figure 6: Architectural Diagram of Adapted Video MAE V1 Structure

The transition from 1024×1024 to 1735×1735 resolution increases the computational burden by a factor of approximately $2.87 \times$ ($1735^2/1024^2 \approx 2.87$) in terms of spatial dimensions alone. When accounting for the increased number of attention operations required in transformer architectures, the actual computational complexity increases super-linearly. For a standard transformer with self-attention, complexity scales as $O(n^2d)$ where n is the sequence length (number of patches) and d is the embedding dimension. With 16×16 patches, the 1735×1735 image yields approximately 11,772 patches compared to 4,096 patches for 1024×1024 images, resulting in an $8.2 \times$ increase in attention computation.

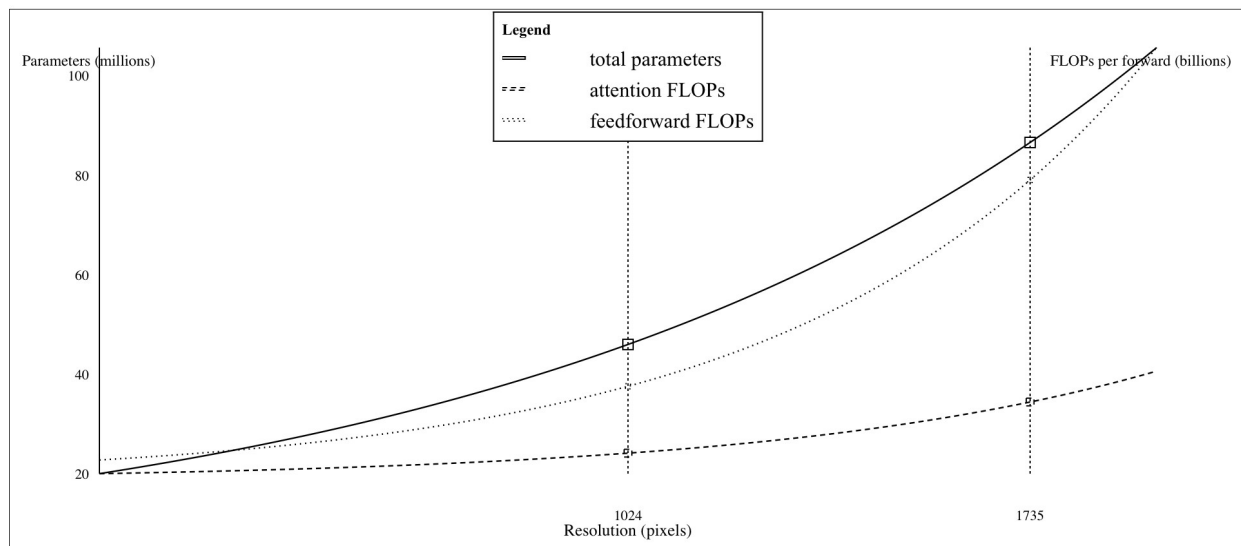


Figure 7: Log-Scale Graph Showing Model Size And Computational Requirements vs. Input Resolution

Due to exponential increases in model size and memory requirements with larger resolutions, sharding strategies have become necessary for implementation. A single NVIDIA RTX 6000 Ada GPU with 48GB VRAM cannot accommodate the full model at native resolution along with the necessary optimizer states and activations required for training. This memory constraint drives the adoption of distributed training approaches detailed in Section 4. The sharding implementation must carefully balance computation across devices while minimizing inter-GPU communication overhead, which can become a bottleneck for high-resolution processing.

Part 4: Distributed Training Strategies for Large-Scale Radar Models

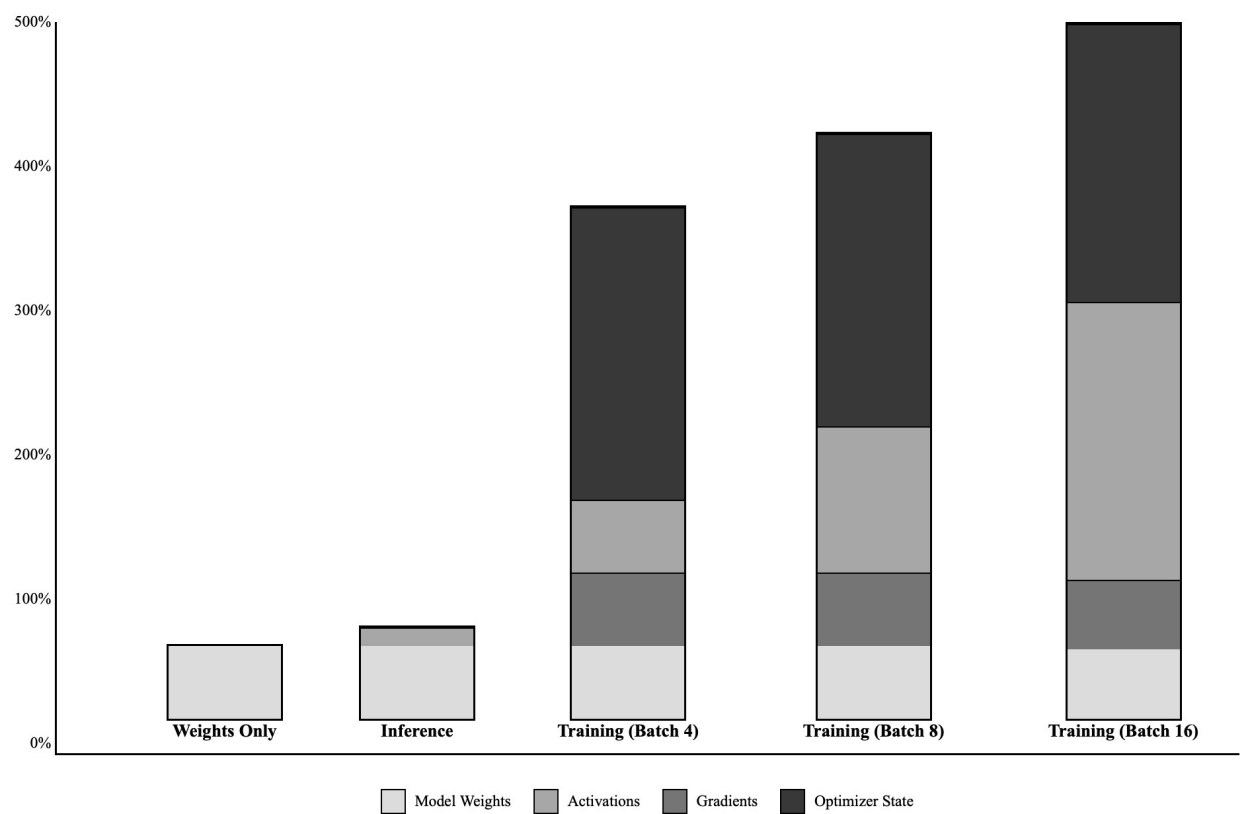


Figure 8: Memory/VRAM Usage for AI Models as a Percentage of the Model Weights

Distributed training has become essential for modern deep learning as models grow beyond the capacity of individual GPU devices(see Figure 8 for example). This section provides a comprehensive analysis of four primary parallelism strategies: data parallelism, model parallelism, pipeline parallelism, and tensor parallelism. Each approach offers distinct trade-offs between computational efficiency, memory utilization, and implementation complexity. Understanding these trade-offs is critical for selecting appropriate strategies for radar processing applications, where model sizes frequently exceed 10 billion parameters and input data dimensions are large.

Part 4a: Data Parallelism via Scaling Through Batch Distribution

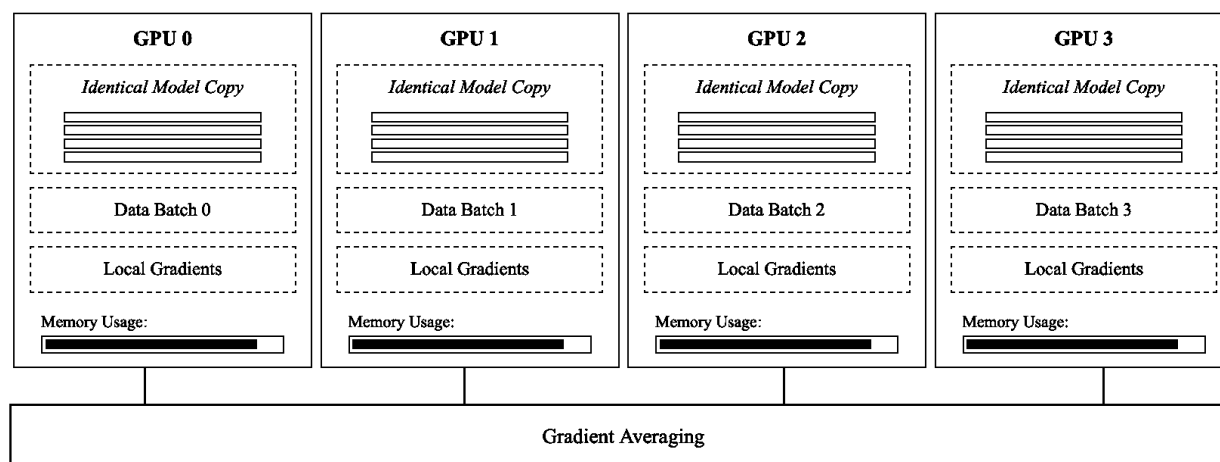


Figure 9: Data Parallelism Schematic Showing 4 GPUs

In data parallelism, the complete model is replicated across multiple GPU devices. This replication ensures that each GPU maintains an identical copy of all model parameters, enabling independent processing of different data subsets. The parallelization occurs at the data level: each GPU receives a distinct batch of training examples and performs complete forward and backward passes independently. This independence during the computation phase is the key advantage of data parallelism, as GPUs do not need to communicate during the forward and backward passes, only during gradient synchronization.

The workflow proceeds as follows: (1) Each GPU processes its assigned data batch through the complete model, computing activations at each layer. (2) Loss is calculated using the model's predictions and ground truth labels. (3) Backpropagation computes gradients with respect to all model parameters locally on each GPU. (4) Gradients are synchronized across all GPUs through an all-reduce operation, typically implemented using efficient collective communication algorithms such as ring all-reduce. (5) The averaged gradients are used to update model parameters, ensuring all GPUs maintain identical model states. (6) The process repeats for the next iteration.

This approach maintains identical model states across all GPUs while enabling accelerated data processing proportional to the number of available GPUs. Ideally, using N GPUs provides $N\times$ speedup in processing throughput, though actual speedups are somewhat lower due to gradient synchronization overhead. The effective batch size is multiplied by the number of GPUs, which can improve training stability and convergence for some models but may require learning rate adjustments to maintain optimal training dynamics.

The primary limitation of data parallelism is memory redundancy. Since each GPU stores a complete model copy, data parallelism does not help when a model is too large to fit on a single GPU. For the 13 billion parameter radar models being developed, each full precision copy requires approximately 52GB (13B parameters $\times$ 4 bytes/parameter), exceeding the 48GB capacity of individual RTX 6000 Ada GPUs. Even with mixed-precision training reducing per-parameter storage to 2 bytes, the model alone requires 26GB, leaving insufficient memory for gradients, optimizer states, and activations.

Part 4b: Model Parallelism via Layer-wise Distribution Across Devices

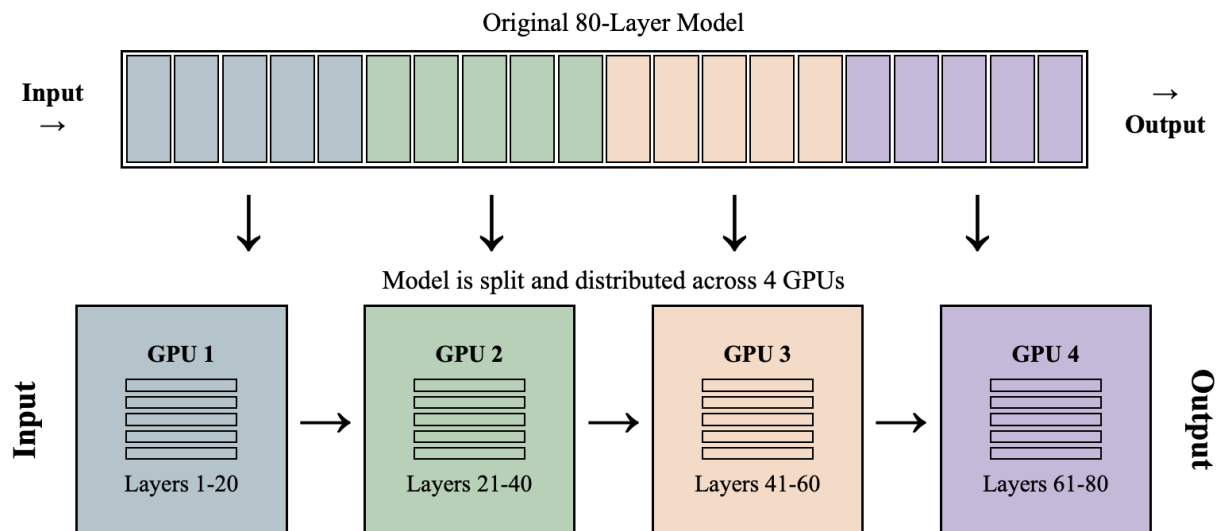


Figure 10: Model Parallelism Diagram for an 80-Layer Network Split Across 4 GPUs

Model parallelism addresses the memory limitation of data parallelism by partitioning network layers across multiple GPU devices. Unlike data parallelism where each GPU has the complete model, model parallelism distributes the model itself across devices. For example, in a 60-layer transformer architecture distributed across three GPUs, layers 1-20 reside on GPU 1, layers 21-40 on GPU 2, and layers 41-60 on GPU 3. This partitioning strategy effectively divides the memory burden, enabling training of models that would not fit on any single device.

Data flows sequentially through these partitioned segments in a pipeline fashion. During forward propagation, input data is processed through layers 1-20 on GPU 1, producing intermediate activations. These activations are transferred to GPU 2 via high-speed interconnects (such as NVLink or PCIe), where they are processed through layers 21-40. The process continues to GPU 3 for final layers 41-60, which produce the model's output. During backpropagation, gradients flow in reverse order: GPU 3 computes gradients for layers 41-60, passes gradient information to GPU 2 for layers 21-40, and finally to GPU 1 for layers 1-20.

Processing Timeline

	Timestep 1	Timestep 2	Timestep 3	Timestep 4
GPU 1	Active	Idle	Idle	Idle
GPU 2	Idle	Active	Idle	Idle
GPU 3	Idle	Idle	Active	Idle
GPU 4	Idle	Idle	Idle	Active

Memory Utilization

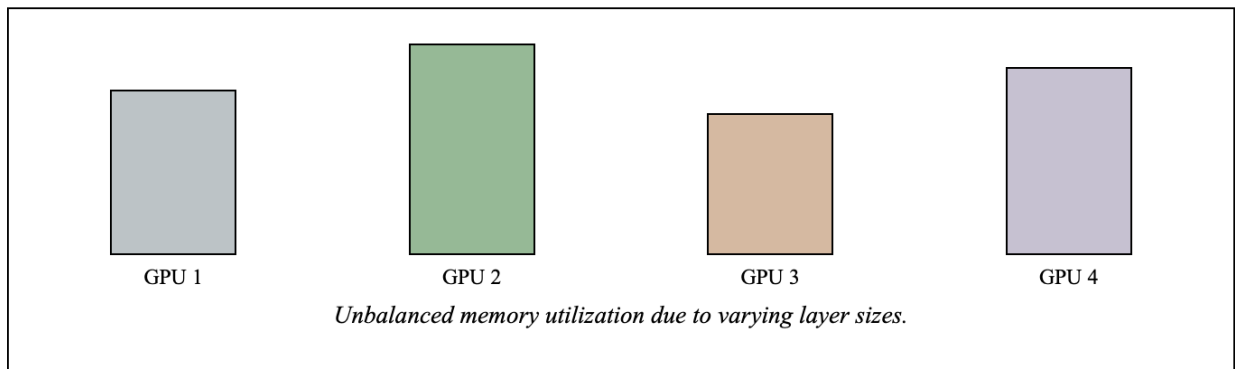


Figure 11: GPU Utilization Over Time For Model Parallelism With 4 GPUs

A significant limitation of this approach is GPU underutilization due to the sequential nature of computation. With N GPUs, each device is active only $1/N$ of the total training time, as processing cannot begin on GPU $i+1$ until GPU i completes its computation. For a 4-GPU configuration, each GPU operates at only 25% utilization, with three-fourths of computational resources remaining idle at any given moment. This inefficiency worsens as more GPUs are added, for example with 10 GPUs, each device would be utilized only 10% of the time, resulting in severe waste of computational resources.

Another challenge is communication overhead. Each layer boundary requires transferring activation tensors between GPUs. For large models with high-dimensional intermediate representations, these transfers can become bottlenecks, particularly when using slower interconnects like PCIe rather than specialized high-bandwidth connections like NVLink. The communication volume scales with the activation size, which for vision transformers can be substantial (e.g., $4096 \text{ tokens} \times 1024 \text{ dimensions} \times 4 \text{ bytes} = 16\text{MB}$ per activation tensor). Multiple such transfers per iteration can significantly impact overall training time.

Despite these limitations, model parallelism remains valuable when models exceed single-device capacity and other parallelism strategies are not applicable. It provides a straightforward conceptual model and is relatively simple to implement compared to more sophisticated approaches. For extremely deep networks (hundreds or thousands of layers), model parallelism offers a natural partitioning strategy aligned with the model's architectural structure.

Part 4c: Pipeline Parallelism via Improving Utilization Through Micro-batching

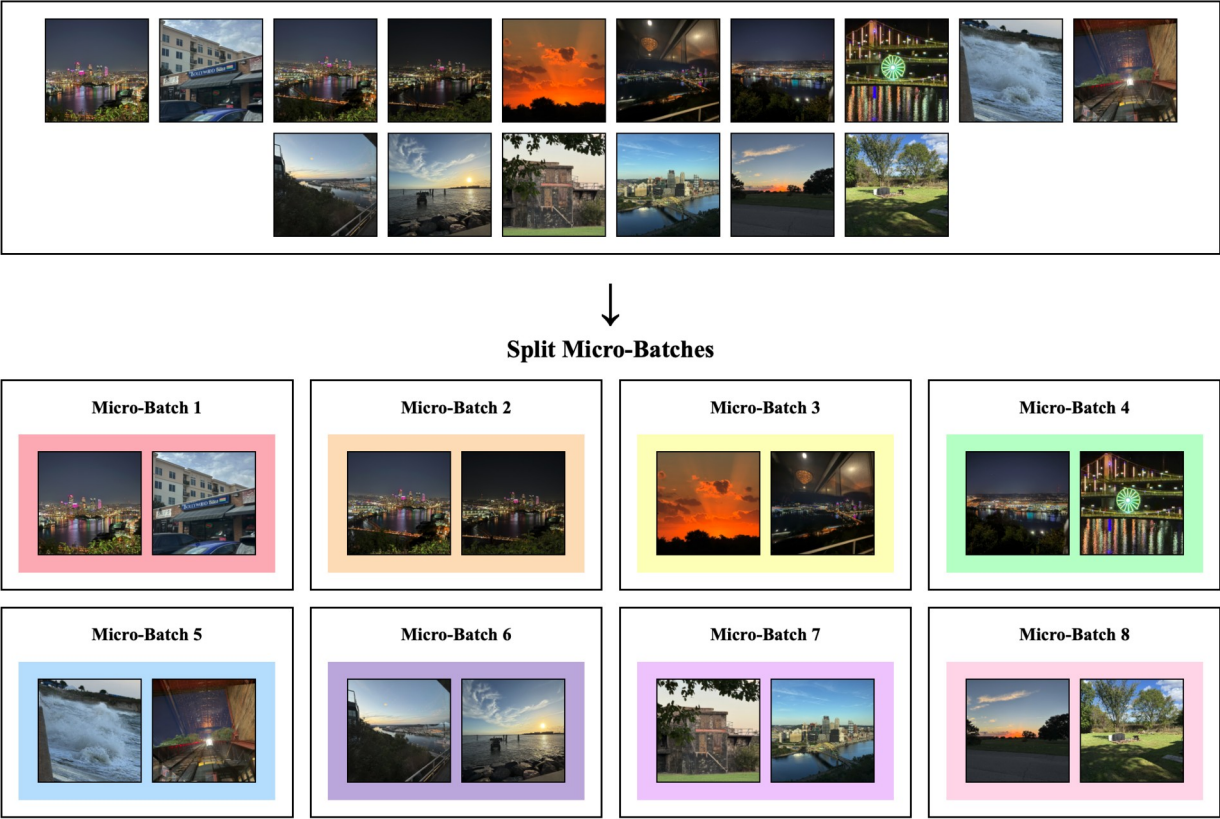


Figure 12: Batch Splitting to Create Micro-Batch Example

Pipeline parallelism addresses the utilization inefficiencies inherent in standard model parallelism through a technique called micro-batching. Rather than processing a single large batch sequentially through the partitioned model, pipeline parallelism divides each training batch into smaller micro-batches. By immediately initiating processing of subsequent micro-batches as soon as the first micro-batch advances to the next GPU, idle time is substantially reduced. This overlapping computation enables more efficient GPU utilization and improved throughput while maintaining the same total batch size.

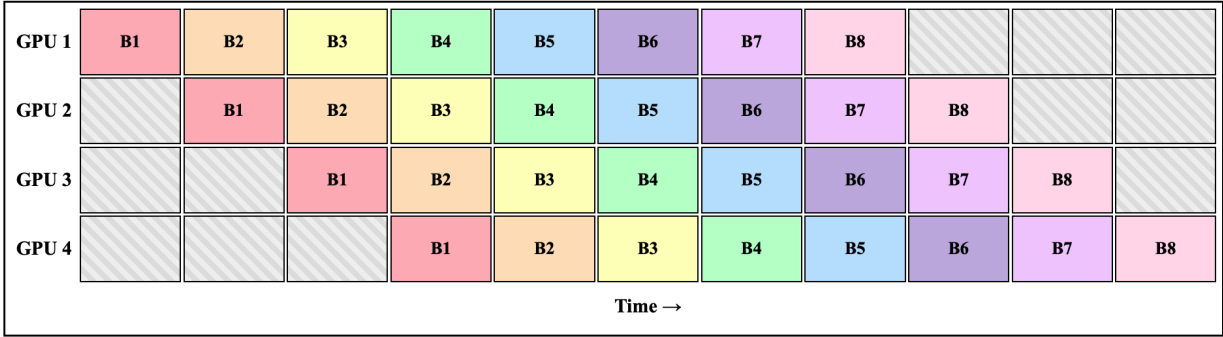


Figure 13: Pipeline Parallelism Visualization Showing 4 GPUs Processing 8 Micro-Batches

The pipeline operates as follows: (1) The training batch is split into M micro-batches, where M is typically $4\text{--}8\times$ the number of GPUs. (2) GPU 1 begins processing micro-batch 1 through its assigned layers. (3) As soon as GPU 1 completes micro-batch 1 and sends results to GPU 2, GPU 1 immediately begins processing micro-batch 2. (4) Meanwhile, GPU 2 processes micro-batch 1 through its layers. (5) This pattern continues, creating a pipeline where all GPUs eventually process different micro-batches simultaneously. (6) Gradients from all micro-batches are accumulated before performing a weight update, ensuring mathematical equivalence to processing the full batch.

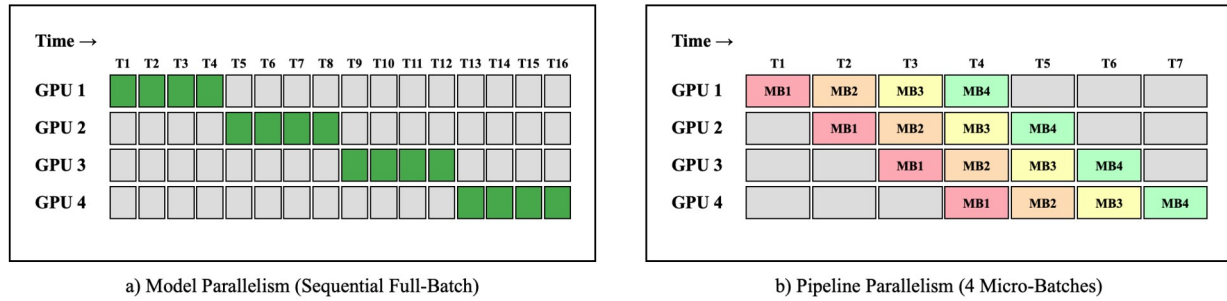


Figure 14: Model v. Pipeline Parallelism. Model Parallelism Takes Nearly Twice as Long in This Configuration

Pipeline parallelism significantly improves GPU utilization compared to basic model parallelism. In the steady state (after the pipeline fills), all GPUs can be active simultaneously, each processing different micro-batches at different pipeline stages. However, utilization is not perfect due to 'pipeline bubbles', the idle periods at the beginning and end of each batch where the pipeline fills and drains. The efficiency improves with more micro-batches relative to the number of GPUs. With M micro-batches and N pipeline stages, the theoretical efficiency is $M/(M+N-1)$. For example, with 32 micro-batches and 4 stages, efficiency is $32/35 \approx 91.4\%$.

A practical challenge with pipeline parallelism is achieving balanced computation across pipeline stages. If one stage takes significantly longer than others, it becomes a bottleneck that limits overall throughput. For transformer models with uniform layer structure, achieving balance is relatively straightforward since each GPU can host approximately the same number of layers. However, models with heterogeneous architectures (e.g., combining convolutional and transformer layers) require careful profiling and manual tuning to distribute computation evenly. Imbalanced pipelines lead to some GPUs waiting idle while others complete their work.

Memory consumption patterns differ between pipeline and model parallelism. While each GPU still stores only a subset of model parameters, pipeline parallelism requires storing multiple micro-batches worth of activations simultaneously - one for each micro-batch currently in flight through that pipeline stage. This increased activation memory can partially offset the memory savings from model distribution. Careful selection of micro-batch size is necessary to balance utilization benefits against memory constraints.

Part 4d: Tensor Parallelism via Intra-layer Distribution

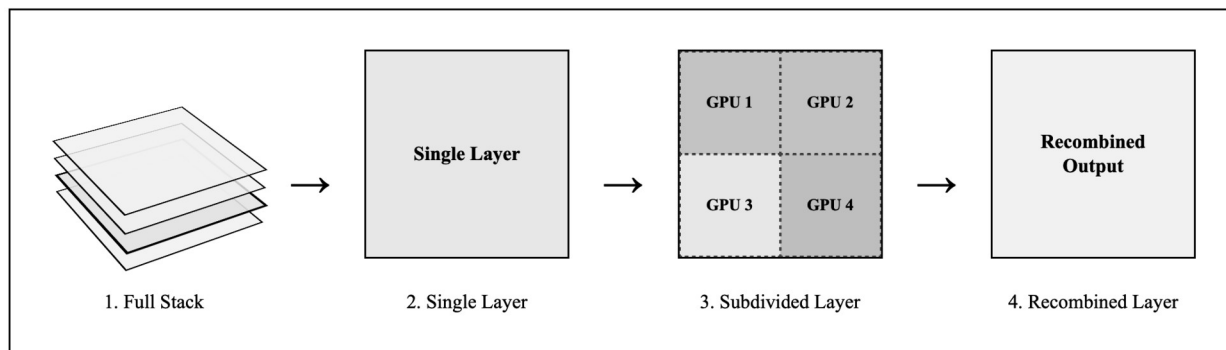


Figure 15: Tensor Parallelism Schematic Showing a Single Transformer Layer Split Across 4 GPUs

Tensor parallelism represents the most sophisticated distribution strategy, particularly suited for large-scale training operations. Rather than partitioning by layers (model parallelism) or data (data parallelism), tensor parallelism divides individual layers across GPUs at the level of matrix operations. Each layer's weight matrices and computations are split across devices, with multiple GPUs cooperating to compute each layer's output. This fine-grained parallelism enables efficient handling of wide architectures with extensive parameter counts per layer - a common characteristic of modern large language models and vision transformers.

The key insight behind tensor parallelism is that many neural network operations involve matrix multiplications that can be partitioned along their dimensional axes. For example, consider a linear layer computing $Y = XW$ where X is the input ($\text{batch_size} \times \text{input_dim}$) and W is the weight matrix ($\text{input_dim} \times \text{output_dim}$). The weight matrix W can be split column-wise across N GPUs: $W = [W_1, W_2, \dots, W_n]$. Each GPU computes a portion of the output: $Y_i = XW_i$, producing $(\text{batch_size} \times \text{output_dim}/N)$ results. These partial results are then combined through inter-GPU communication to produce the complete output.

For transformer architectures, tensor parallelism is applied to multiple components within each layer. Multi-head attention mechanisms naturally partition across attention heads - with 32 attention heads and 4 GPUs, each GPU computes 8 heads independently. The feedforward networks (FFNs) that comprise the majority of transformer parameters are parallelized by splitting both the up-projection and down-projection matrices. Careful placement of communication operations ensures mathematical correctness while minimizing synchronization overhead.

This approach proves especially effective for models with substantial width but limited depth, where individual layers may exceed single-GPU memory capacity. For instance, a single transformer layer with 4096 embedding dimension and 16384 hidden dimension in its FFN contains approximately 100M parameters ($4096 \times 16384 \times 2 \approx 134\text{M}$). At FP16 precision, this requires 268MB just for weights, and significantly more when including gradients and optimizer states. Tensor parallelism allows such layers to be distributed across multiple devices, making otherwise impossible model configurations feasible.

The communication requirements of tensor parallelism differ fundamentally from model and pipeline parallelism. While model/pipeline parallelism requires communication only at layer

boundaries, tensor parallelism requires communication within each layer - specifically, after each parallelized operation. However, the communication volume per operation is typically smaller (partial activation tensors rather than full activation volumes). The frequency vs. volume trade-off favors tensor parallelism when high-bandwidth, low-latency interconnects are available (e.g., NVLink, InfiniBand), but may be less favorable with slower interconnects (PCIe).

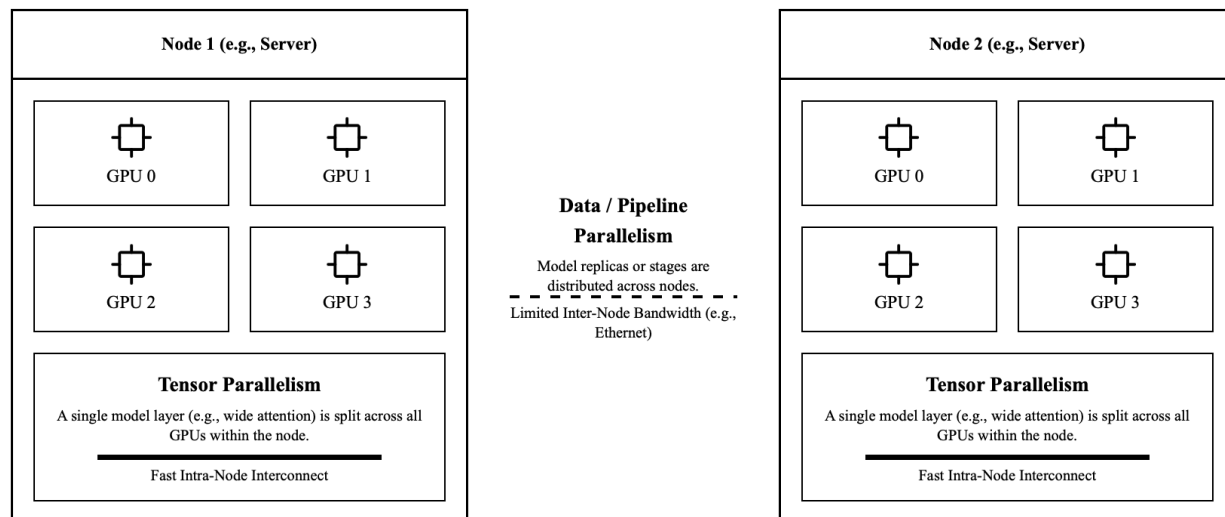


Figure 16: An Example of 2D Parallelism; Inter-Layer Tensor Parallelism with Inter-Node Data Parallelism

In practice, modern distributed training often combines multiple parallelism strategies. A common pattern is 2D parallelism: tensor parallelism within multi-GPU nodes (leveraging fast intra-node interconnects) and data or pipeline parallelism across nodes (where inter-node bandwidth is more limited). This hybrid approach allows models to scale to hundreds or thousands of GPUs while maintaining reasonable efficiency. For the radar processing models under development, tensor parallelism is being evaluated for handling the wide attention mechanisms required by high-resolution inputs.

Part 5: Distributed Training Libraries and Frameworks

Multiple frameworks enable distributed training of large neural networks, each with different trade-offs. This section evaluates PyTorch Distributed, Microsoft DeepSpeed, NVIDIA Megatron, Hugging Face Accelerate, and Google JAX for radar model training.

PyTorch offers two primary strategies. DDP replicates the complete model across GPUs, providing near-linear scaling for models fitting in single-GPU memory. FSDP shards parameters, gradients, and optimizer states across devices, enabling larger models at the cost of increased communication overhead through all-gather and reduce-scatter operations.

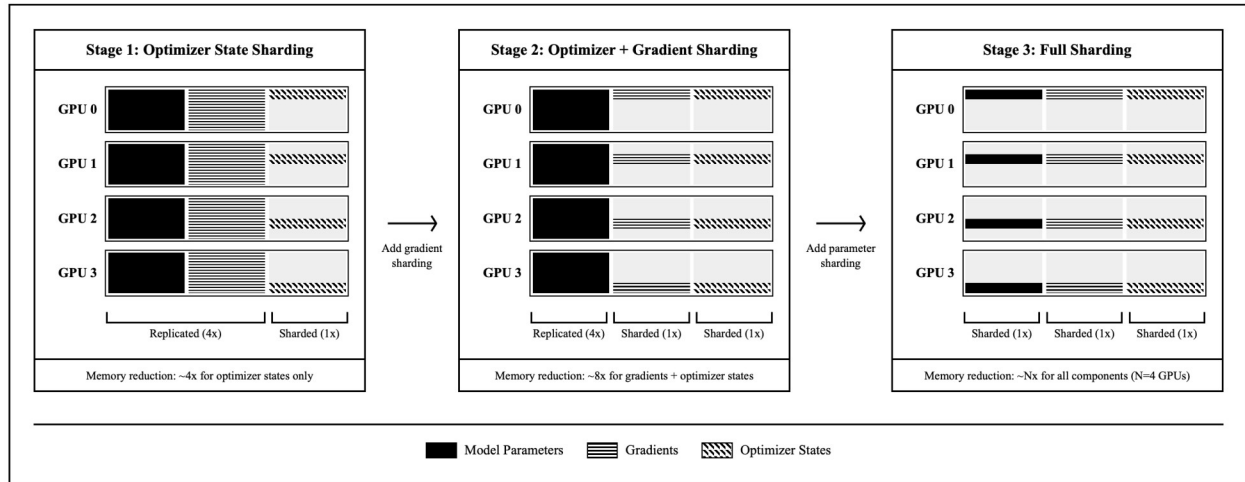


Figure 17: A summary of different ZeRO stages(1-3)

Microsoft DeepSpeed's Zero Redundancy Optimizer (ZeRO) provides three progressive stages: Stage 1 shards optimizer states only; Stage 2 adds gradient sharding; Stage 3 shards model parameters for maximum memory efficiency. Stage 2 balances memory savings and speed for most applications. DeepSpeed also supports CPU/NVMe offloading for extremely large models.

NVIDIA Megatron-LM specializes in massive transformer language models through tensor and pipeline parallelism. Its language modeling focus and architecture-specific optimizations make it less suitable for vision transformers used in radar tasks.

Hugging Face Accelerate provides a unified interface abstracting distributed training complexity. Researchers write device-agnostic PyTorch code that automatically adapts to single-GPU, multi-GPU, or multi-node setups. Configuration-driven YAML files enable switching between DDP, FSDP, and DeepSpeed backends without code changes. This emerged as the preferred solution for this research project, combining ease of use with strong DeepSpeed backend support.

Google JAX uses functional programming and JIT compilation for aggressive optimizations. While powerful and used by Google for large-scale training, its steeper learning curve and functional paradigm made PyTorch-based frameworks more pragmatic for this research.

Part 6: Radar Frame Prediction Model

The radar frame prediction model represents the tepid culmination of distributed training infrastructure development and temporal modeling research. This section documents the complete implementation pipeline from raw radar data acquisition through preprocessing, model training, and quantitative/qualitative evaluation. The model processes sequences of high-resolution radar frames captured from maritime environments, learning to predict future radar returns based on historical context. Results demonstrate both promising capabilities in land feature preservation and current limitations in small object detection, providing direction for future research.

Part 6a: Dataset Specs

The radar frame prediction model was trained on multi-pulse length datasets collected specifically for this research. The dataset comprises triplet sequences captured at varying pulse lengths: 60, 65, and 70 game pulls across different modulation schemes (L2, L, and M1). This multi-pulse length approach enables the model to learn robust features across different radar operational parameters rather than overfitting to a single configuration.

The raw radar data was captured at the native resolution of 1735×1735 pixels, representing the full spatial extent of the radar coverage area. This high resolution preserves fine-grained spatial details but presents significant computational challenges for processing. To balance model capacity constraints with spatial fidelity, the data underwent resolution adjustment during preprocessing while maintaining the aspect ratio and relative spatial relationships between radar returns.

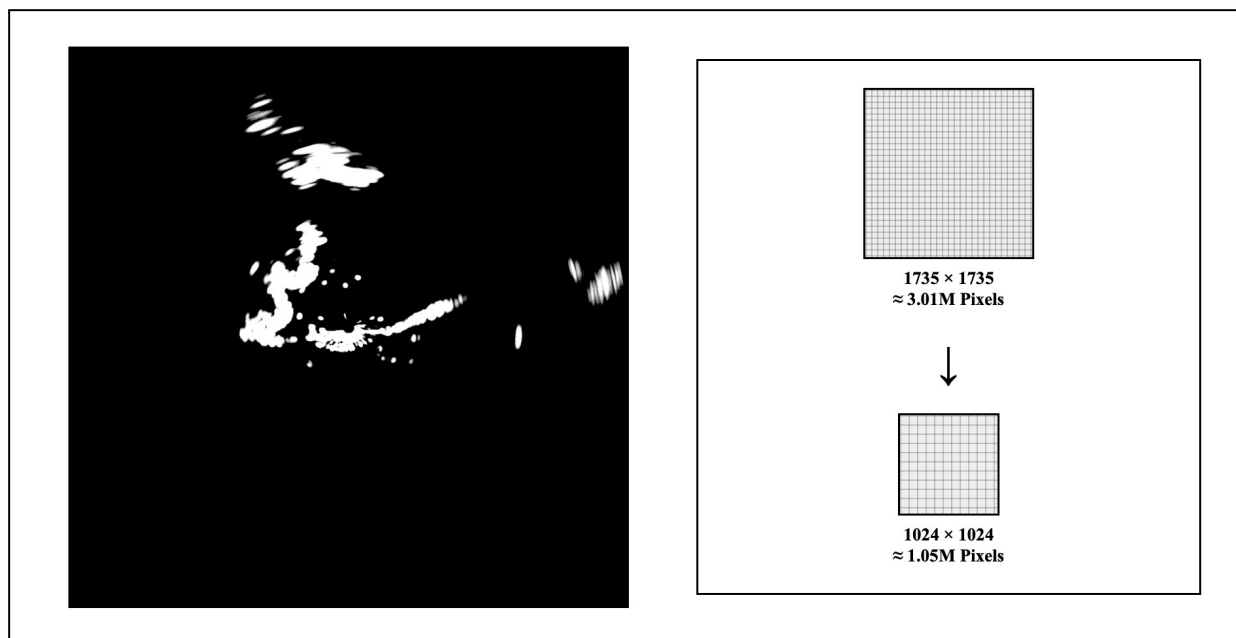


Figure 18: Example Of Radar Data And A View Of Resolution Reduction

Part 6b: Preprocessing Pipeline

Two distinct preprocessing strategies were evaluated to handle the transition from native 1735×1735 resolution to the model's 1024×1024 input size. The first approach employed spatial downsampling, directly resizing the full-resolution radar frame to 1024×1024 pixels using bilinear interpolation. This method preserves the global spatial context but sacrifices some fine-grained detail due to the resolution reduction.

The second preprocessing approach divided each radar frame into four quadrants, effectively creating four separate 1024×1024 samples from each original frame through cropping and slight upscaling. This quadrant-based method maintains higher spatial resolution at the cost of fragmenting the global context. Each quadrant becomes an independent training sample, quadrupling the effective dataset size while preserving more detailed spatial features within each region.

Data augmentation techniques were applied during training to improve model generalization and robustness. The augmentation pipeline included random horizontal and vertical flips to account for directional invariance in radar returns, rotation transformations to handle varying object orientations, color jittering to simulate intensity variations in radar returns, Gaussian blur to model atmospheric effects and sensor noise, and additive noise injection to increase robustness to measurement uncertainty. While these transformations demonstrably improved training curve convergence, visual inspection of model outputs suggested limited improvement in prediction quality for held-out validation samples.

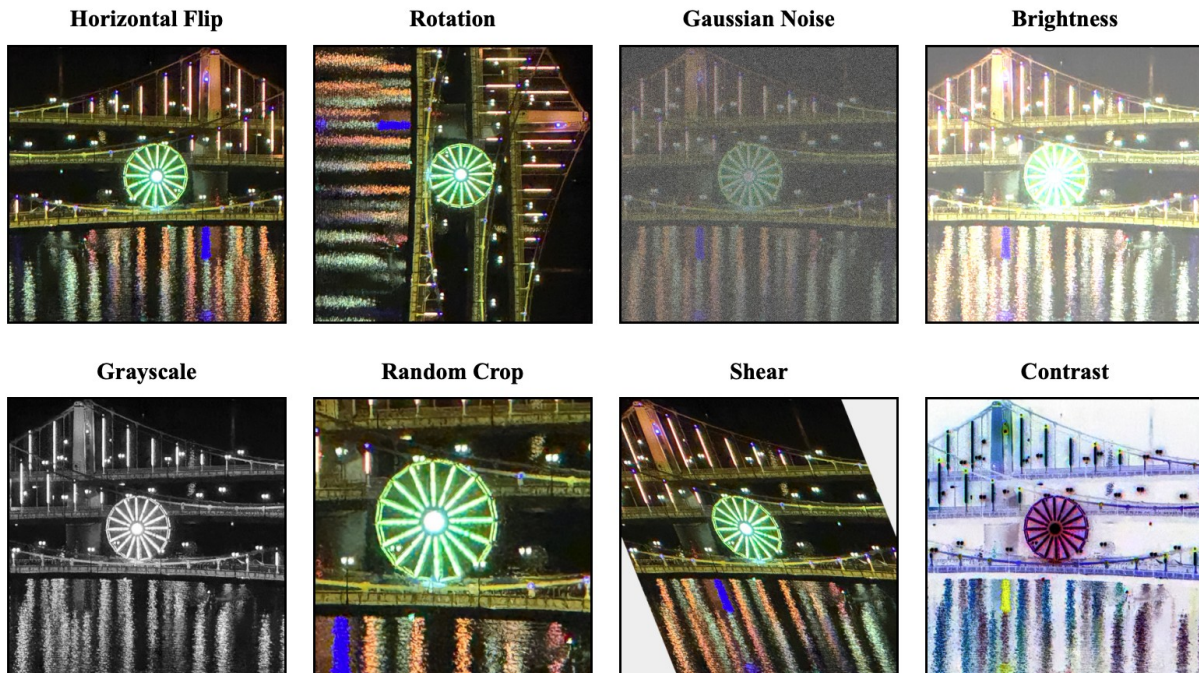


Figure 19: A Selection Of Augmentation Methods For Images

Part 6c: Model Architecture

The model implements a vision transformer-based masked auto-encoding architecture adapted for temporal radar prediction. The design employs a 16×16 pixel patch size with stride matching, decomposing each 1024×1024 input frame into 4096 distinct patch tokens. This patch-based tokenization enables the transformer architecture to operate on manageable sequence lengths while preserving spatial locality through the patch structure.

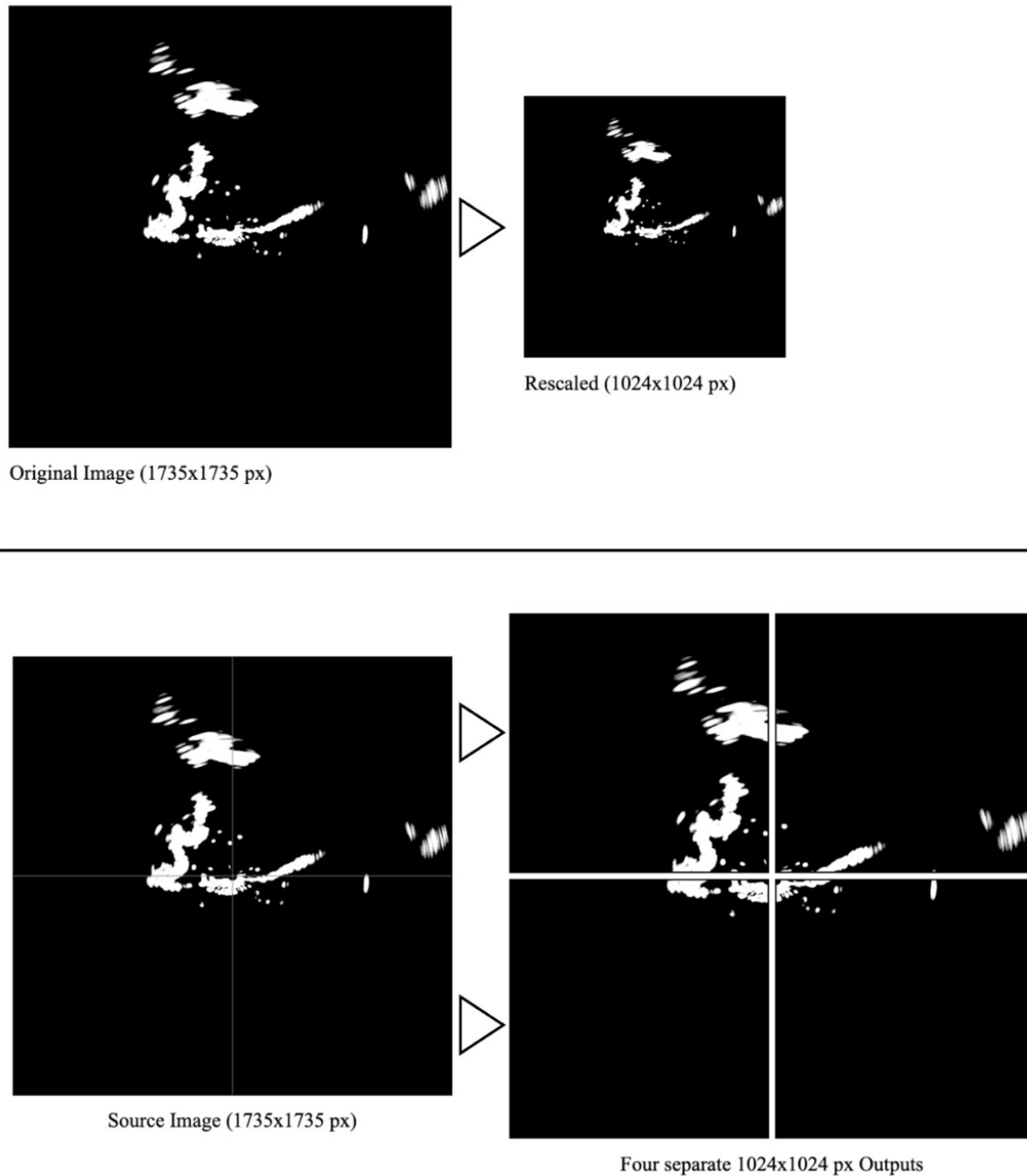


Figure 20: Data Augmentation Pipeline Comparing Two Methods. Quadrant Upscaling Or Full Frame Downscaling.

Positional embeddings are learned rather than fixed, allowing the model to adaptively encode spatial relationships specific to radar data patterns. The architecture incorporates optional random masking of patch tokens controlled via command-line interface parameters, enabling exploration of different masking ratios during training. A lightweight vision transformer encoder processes the unmasked patches, learning compressed representations of the visible radar returns.

The decoder component receives both the encoded patch representations and learned mask tokens corresponding to the missing patches. Multiple transformer decoder blocks reconstruct the full sequence, with each block attending to both visible and masked locations. A linear projection head maps the decoder outputs to per-patch pixel predictions, effectively reconstructing the radar frame at the original resolution. The model accepts a single input frame and produces predictions for the subsequent two frames, requiring the decoder to learn temporal evolution patterns in addition to spatial reconstruction.

Part 6d: Training Configuration

The model was trained using the AdamW optimizer with cosine learning rate decay schedule. This optimization configuration balances fast initial convergence with stable long-term training dynamics. The training infrastructure leveraged Hugging Face Accelerate library with DeepSpeed ZeRO Stage 2 optimization, enabling efficient memory utilization through gradient and optimizer state sharding across available GPU devices.

Training employed FP16 mixed precision to accelerate computation while maintaining numerical stability. Although only a single GPU was utilized for these initial experiments, the DeepSpeed integration was maintained in the codebase to facilitate future scaling to multi-GPU configurations. This forward-looking infrastructure design ensures that the training pipeline can seamlessly transition to distributed training as model sizes and dataset scales increase.

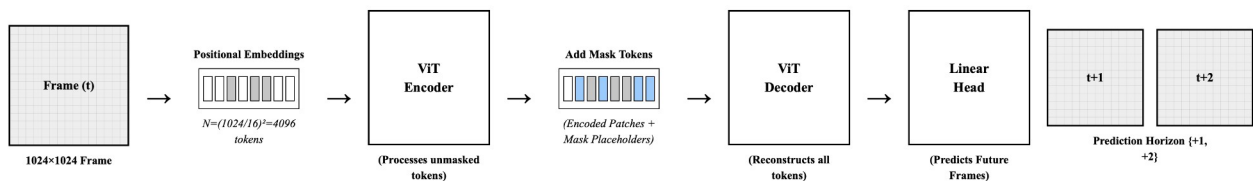


Figure 21: Temporal MAE-style ViT architecture

Part 6e: Loss Function Analysis

Initial training experiments relied exclusively on mean squared error (MSE) loss for frame reconstruction. MSE penalizes large pixel-wise deviations but treats all prediction errors equally regardless of perceptual importance. This loss function resulted in predicted frames that appeared overly smooth and blurred, as the model learned to minimize squared error by averaging over possible futures rather than committing to sharp, well-defined predictions.

To address this blurriness problem, L1 (mean absolute error) loss was incorporated into the training objective with a weighting coefficient of 0.2 relative to the MSE term. L1 loss is less sensitive to outliers than MSE and can encourage sharper predictions by penalizing small errors more uniformly. However, contrary to expectations, the addition of L1 loss yielded worse visual

results, with predicted frames exhibiting reduced accuracy and increased blur compared to the MSE-only baseline.

The validation loss curves indicated faster convergence with the combined loss function, reaching lower loss values in fewer epochs. This apparent contradiction between improved training metrics and degraded visual quality suggests that the loss landscape may not perfectly align with perceptual quality for radar prediction tasks. Further investigation is needed to determine whether extended training duration would allow the L1-augmented model to surpass the MSE baseline, or whether alternative loss formulations such as perceptual losses or adversarial training objectives would better capture the desired prediction characteristics.

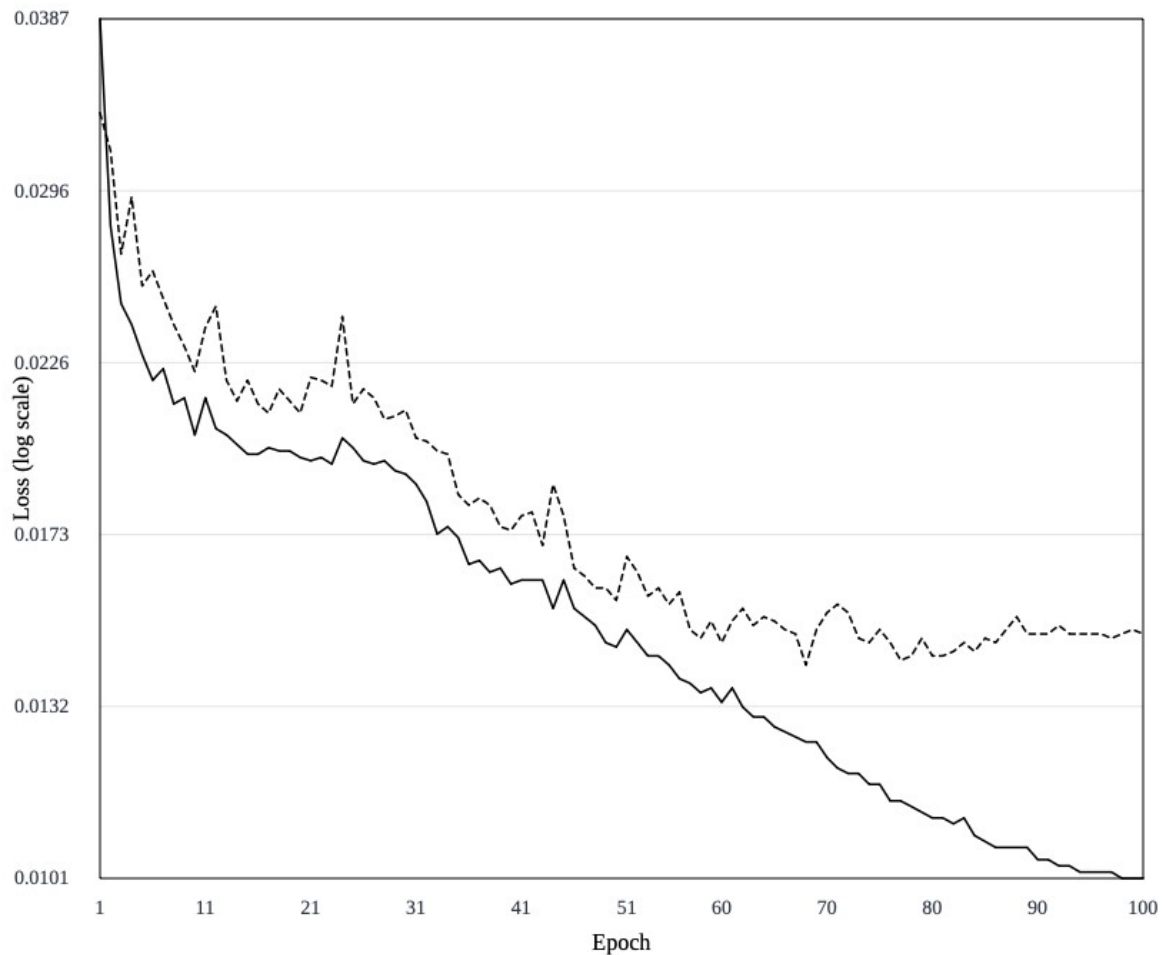


Figure 22: Training & validation loss over epochs. Before data augmentation

Part 6f: Experimental Results

The model demonstrates proficient performance in preserving large-scale land features and maintaining overall spatial structure in predicted frames. When provided with an initial radar frame showing coastal boundaries and geographic landmarks, the model successfully propagates these stationary features into future frames with high fidelity. This capability indicates effective learning of the spatial prior that land masses remain stable across consecutive radar sweeps.

However, significant limitations emerge when evaluating small object detection and tracking performance. The model consistently fails to identify and predict the positions of critical maritime targets such as boats and buoys. These small, mobile objects appear in the input frames as isolated bright pixels or small clusters, but the predicted frames either omit them entirely or represent them as blurred, barely distinguishable artifacts. This failure mode represents a critical deficiency for practical maritime surveillance applications where detecting and tracking vessels is the primary operational requirement.

Training dynamics revealed that model performance plateaued around epoch 60, with training loss continuing to decrease while validation loss flatlined. This divergence between training and validation curves indicates potential overfitting, where the model begins to memorize training examples rather than learning generalizable temporal patterns. The validation set performance ceased improving despite continued optimization, suggesting that the model architecture or training procedure may require modification to achieve better generalization.

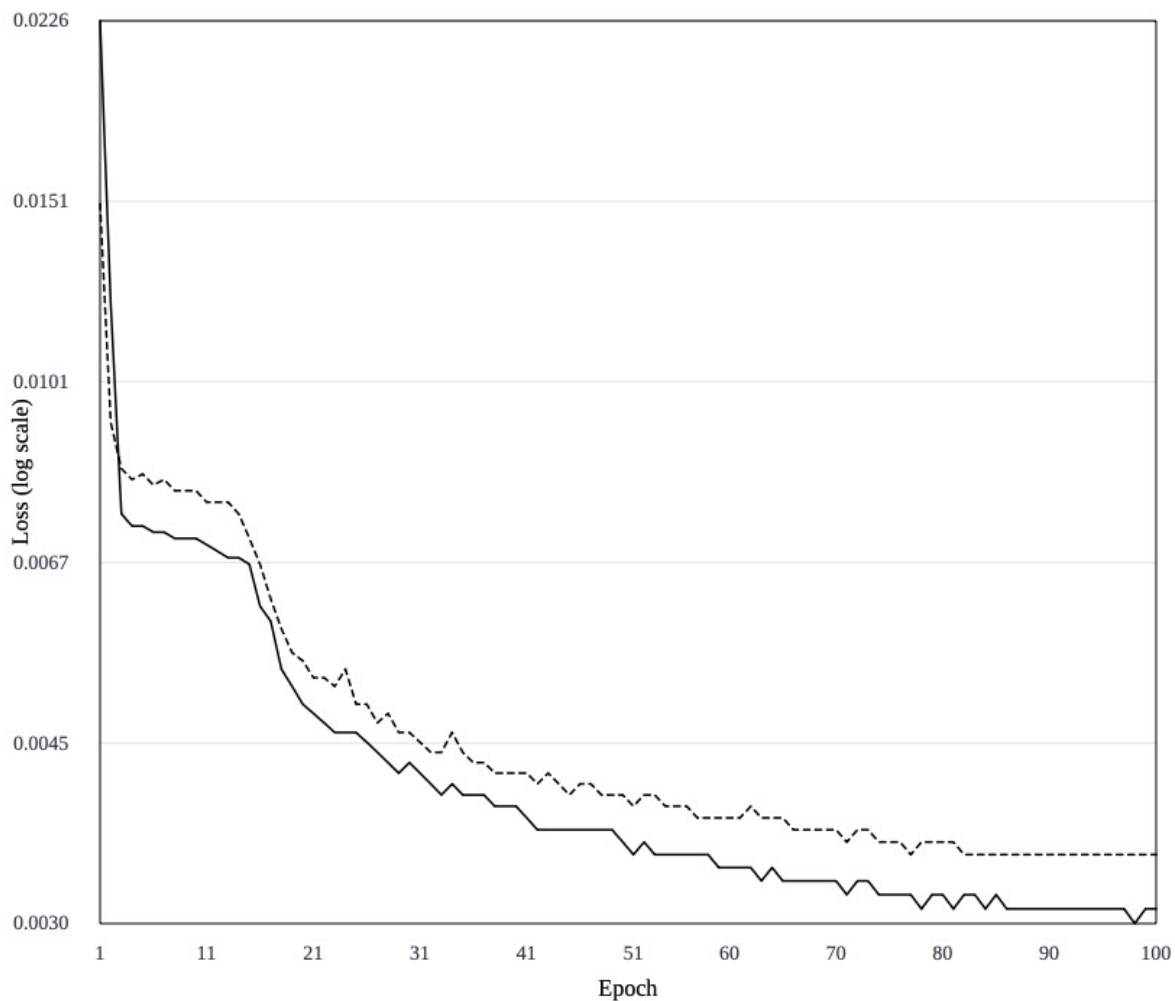


Figure 23: Training & Validation loss over epochs. After data augmentation

Part 7: Undergraduate Research & Agentic Tools

Two undergraduate researchers were recruited and trained to support complementary stages of the research pipeline. This section outlines the methodology – emphasizing modern AI-assisted development – and clarifies the division of responsibilities across data collection, signal-processing implementation, and algorithm development. The training program focused on both technical skill-building and independent problem-solving.

A central contribution of this work is the mandated use of command-line (CLI) agentic development tools powered by large language models (LLMs). Both students were trained on two leading free coding agents – Google’s Gemini CLI and Alibaba’s Qwen Code. These tools enable rapid prototyping and code generation without extensive prior experience in a given programming language, which is especially valuable for students who understand general programming concepts but are unfamiliar with the specific software stack of an ongoing project. In our setting, this approach substantially accelerated development velocity for students still building programming expertise, reducing onboarding time.

Part 7a: Tool Review

Although Gemini and Qwen were introduced to the student first, two premium competitors – OpenAI’s Codex CLI and Anthropic’s Claude Code – consistently produced stronger results. However, both require a \$20/month subscription and are therefore not recommended at the earliest training stage due to cost.

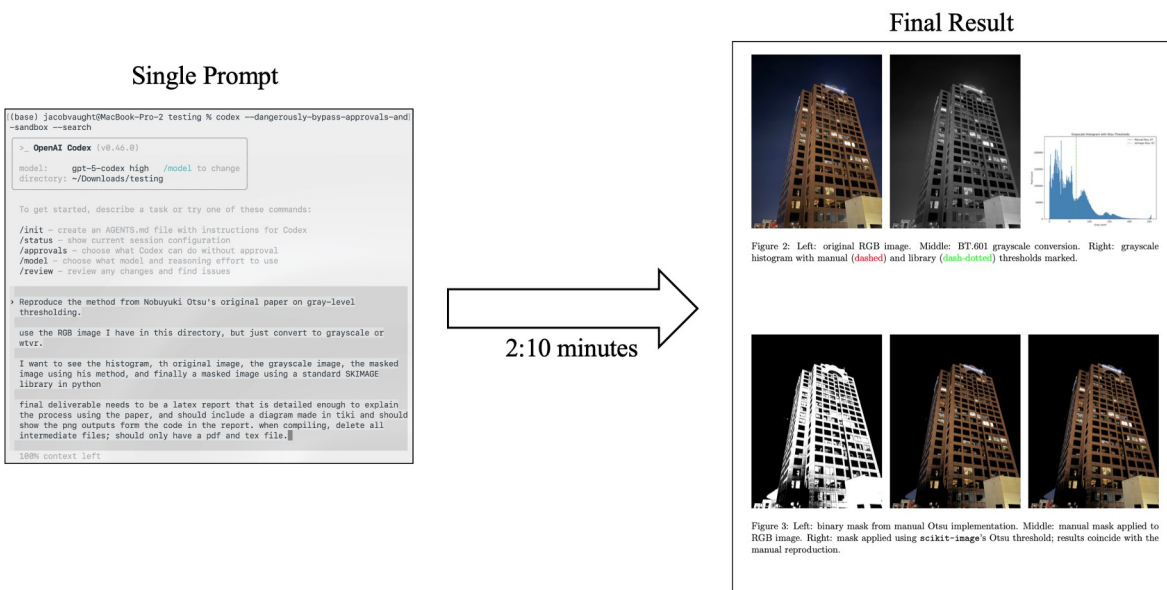


Figure 24: Agentic workflow to solve problem. Creates final report of work completed in under 3 minutes.

Qwen offers a higher usage limit than Gemini (approximately 2×) but lacks robust search, which is a significant handicap given its uneven generalization across languages which means it must rely more heavily on external documentation. It performs well in areas of strength – primarily

Python and its most popular libraries – but it will sometimes halt before completing a task and can enter “loop” states (fixing one error only to reintroduce a previous one). Qwen outperforms the somewhat deprecated GitHub Copilot in autocomplete and small tasks, but it is less suitable for fully autonomous workflows and raises several potential security-exposure concerns due to being run by a foreign country.

Gemini’s search integration is excellent – likely benefiting from Google’s proprietary search – which partly offsets its more modest programming proficiency. The “Flash” model was observed to loop more frequently than Qwen and to exhibit intermittent tool-call failures, at times requiring a full restart. The “Pro” model did not have these issues as often. Gemini did produce more somewhat novel or unexpected solutions than both the paid and free alternatives, possibly due to higher output temperature or superior search augmentation. Finally, both OpenAI and Google offer “secure” variants intended for U.S. government compliance, however these incur additional costs and were not used in this project.

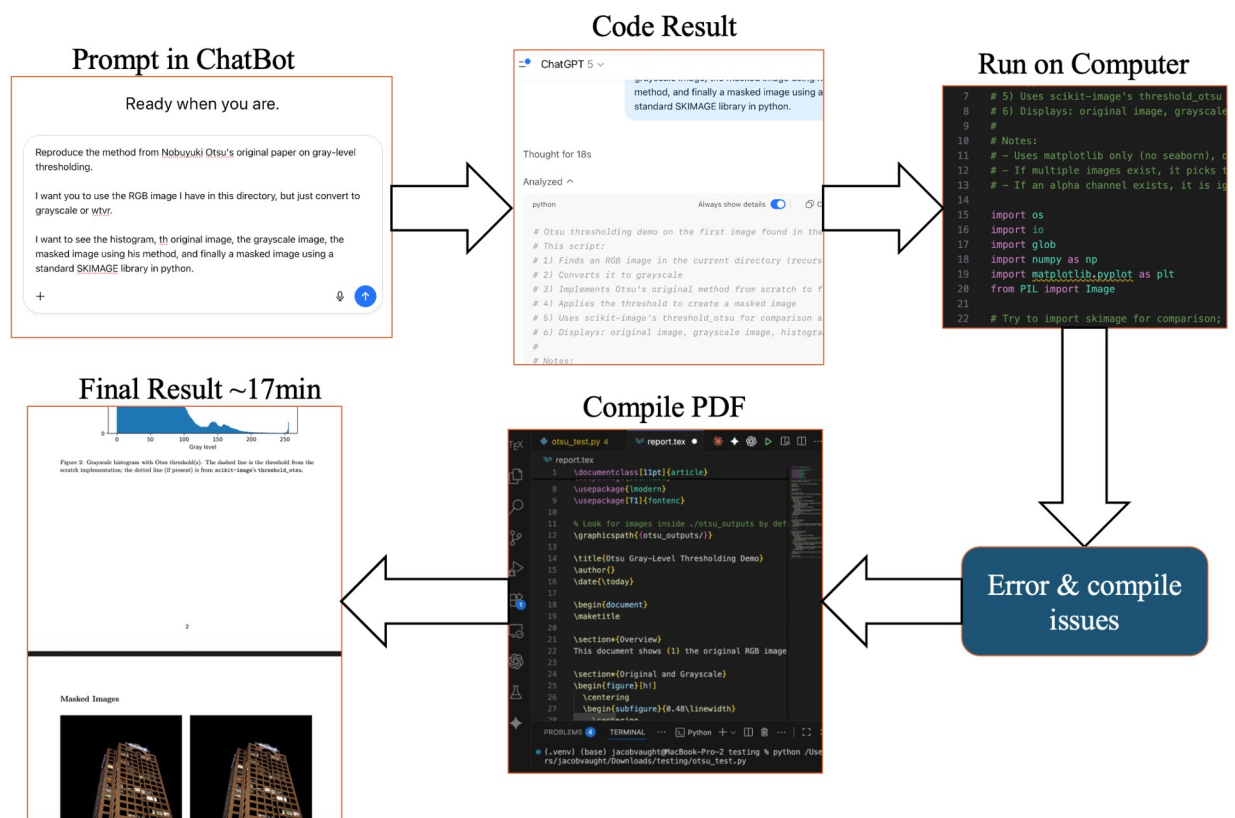


Figure 25: Standard AI-assisted workflow. Approx. 8x as long as Agentic AI workflow

Part 7b: Results

Within the first eight weeks, Sam and Nathan achieved markedly faster turnaround on coding tasks, which we attribute to the structured use of agentic tools. Noted drawbacks included Gemini’s aggressive rate-limiting (both students gravitated toward Gemini, with Qwen hardly being utilized), but overall outcomes were positive.

In a simple benchmark task where agentic models were asked to replicate a research paper, implement and test its results on an image, then generate a LaTeX-formatted PDF output, all models completed the process in under three minutes, with most finishing closer to two. By contrast, the traditional workflow of manually copying and pasting code into a chat or browser interface and debugging it there took around 17 minutes. This represents nearly an 8x reduction in time with agentic tools. While the time savings can compound significantly in repetitive coding tasks, they tend to narrow in more complex workflows with multiple interdependent components.

After the students consistently delivered usable artifacts and learned to prompt these CLIs effectively, they were granted access to ‘premium’ models – namely Codex – with Claude reserved for code cleanup and final reporting due to its restrictive quotas (even tighter than Gemini’s free tier). In contrast to the free tools, Codex and Claude demonstrated credible “autonomous” behavior. For example, when given a research paper and dataset, the models could implement and test multiple ML approaches with minimal user input and generate a well-made latex report with figures and analysis. This enabled parallelization of research directions and abstraction of lower-level implementation work.

Part 8: Signal Processing via Clustering and Temporal Tracking – Samuel

Parallel to deep learning approaches, classical signal processing techniques provide complementary capabilities for radar data analysis. This code and this section of the report was developed and partially written by Samuel Cancilla.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) provides unsupervised segmentation of radar returns into distinct objects. ST-DBSCAN extends this to the temporal domain, enabling object tracking across multiple radar sweeps. These algorithms offer interpretable, computationally efficient alternatives to end-to-end neural network approaches, with particular advantages for real-time applications and scenarios with limited training data.

Part 8a: DBSCAN Algorithm Implementation

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) was implemented as the foundation for unsupervised radar blob detection. The algorithm operates by identifying regions of high point density in the radar data, grouping together pixels that satisfy local density criteria while marking isolated points as noise. This density-based approach proves particularly well-suited for radar data where targets manifest as coherent clusters of elevated intensity pixels against a sparser background.

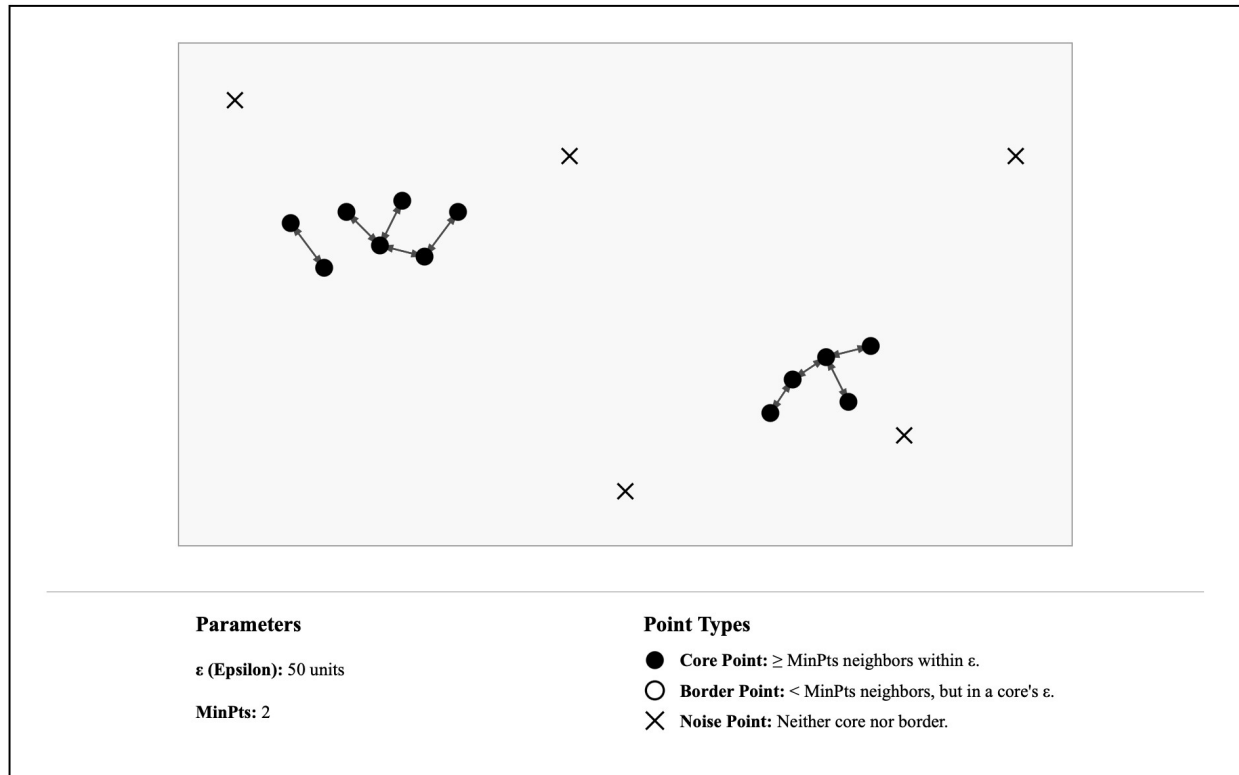


Figure 26: DBSCAN neighborhood and point types

The algorithm operates through iterative neighborhood expansion. It begins by selecting an arbitrary unvisited point from the dataset. For this seed point, the algorithm constructs a circular neighborhood of radius epsilon (ϵ), examining all points falling within this spatial window. If the neighborhood contains at least MinPts points (typically set to 3-5 for radar applications), the seed point is classified as a core point and a new cluster is initiated.

Cluster expansion proceeds by recursively examining the neighborhoods of all newly added core points. Each neighboring point within epsilon distance is added to the cluster, and if this neighbor is itself a core point (has MinPts neighbors), its neighbors are also examined. This process continues until no additional core points can be reached, defining the full spatial extent of a density-connected cluster. Points that fall within the neighborhood of a core point but do not themselves have sufficient neighbors are classified as border points. Points with no core points in their neighborhood are classified as noise and remain unclustered.

The algorithm's unsupervised nature eliminates the need for labeled training data, a significant advantage when working with new radar datasets where ground truth annotations may be unavailable or expensive to acquire. DBSCAN automatically determines the number of clusters present in each frame based on the observed point density patterns, adapting to scenes with varying numbers of targets without requiring prior specification.

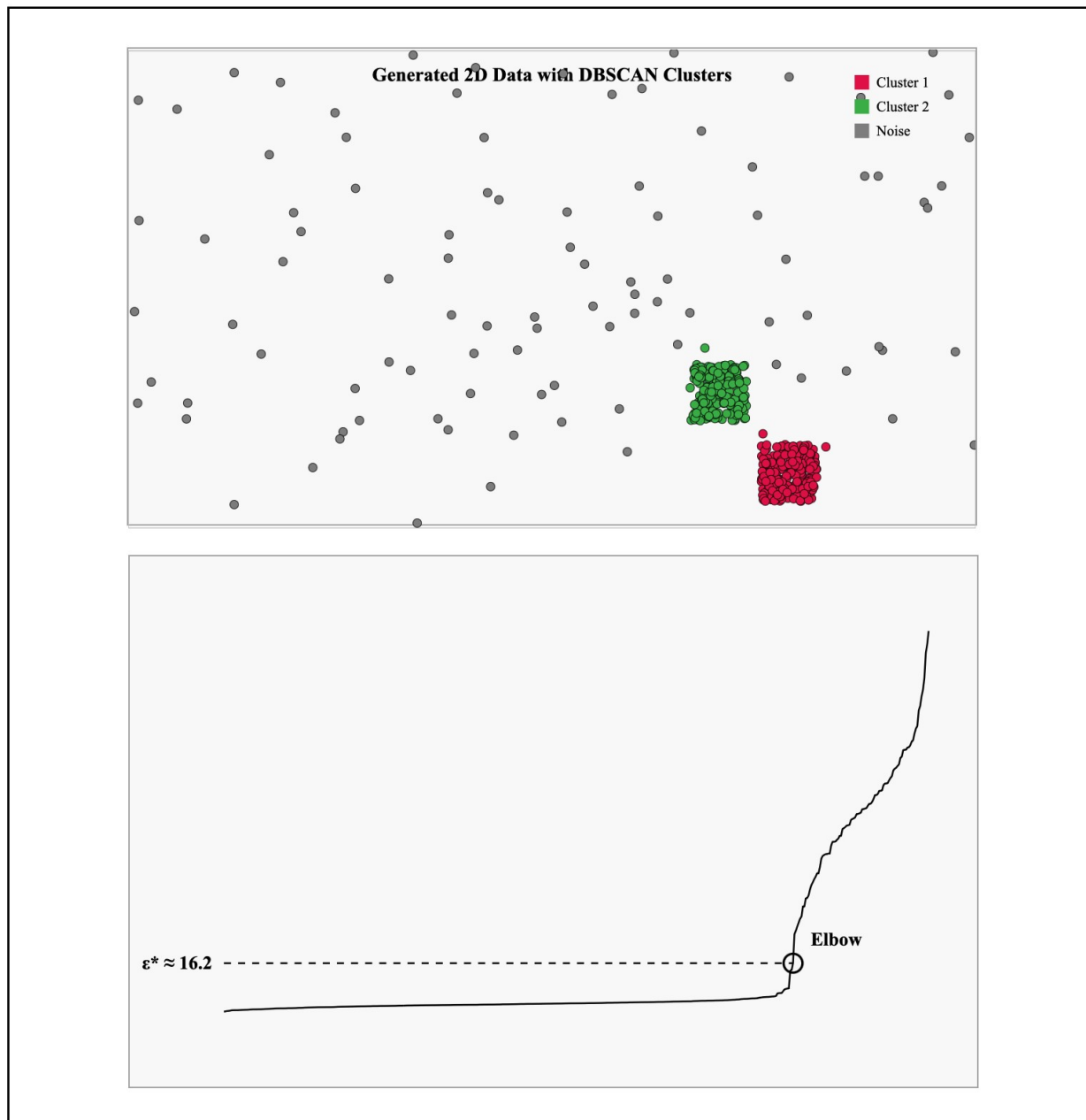


Figure 27: Epsilon Selection Via K-Distance Elbow Algorithm

Implementation challenges primarily centered on parameter selection. The epsilon radius must be large enough to bridge small gaps in radar returns from the same physical object but small enough to avoid merging distinct nearby targets. MinPts similarly requires careful tuning: too low allows noise pixels to form spurious clusters, while too high fragments legitimate targets into multiple smaller clusters. Extensive empirical testing across diverse radar scenes informed the selection of default parameter ranges suitable for maritime radar applications.

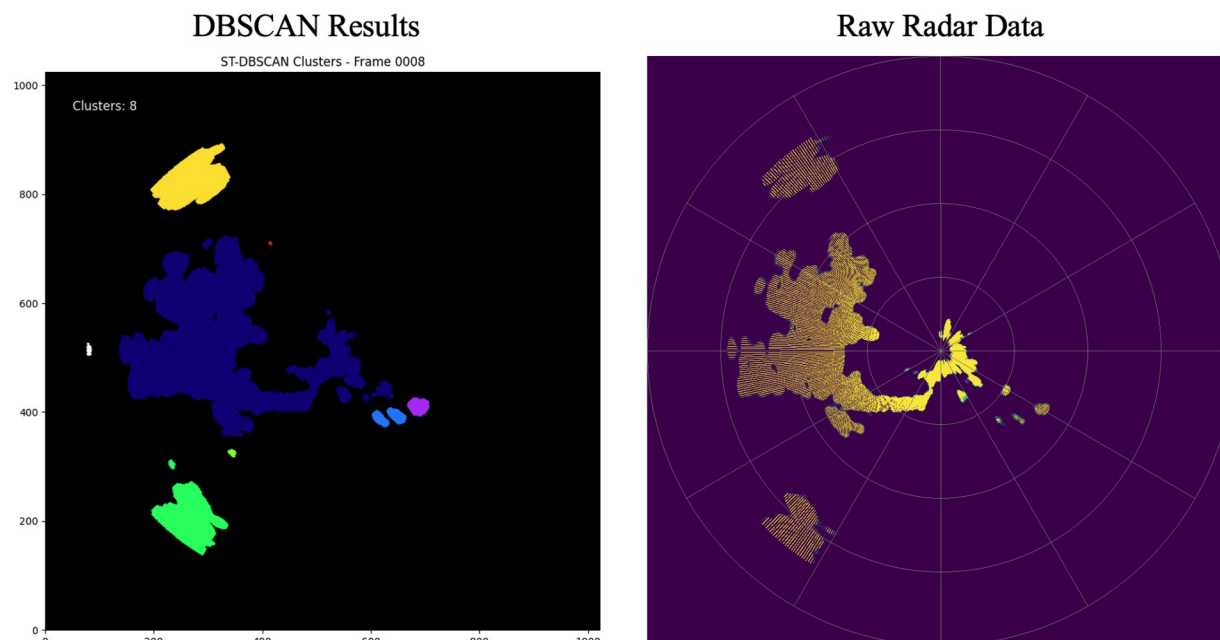


Figure 28: DBSCAN results on Radar Data

Part 8b: Spatio-Temporal DBSCAN (ST-DBSCAN)

ST-DBSCAN extends the spatial clustering capabilities of DBSCAN into the temporal domain, enabling tracking of radar blobs across multiple consecutive frames. While standard DBSCAN operates on individual radar sweeps, ST-DBSCAN treats sequential frames as a 3D space-time volume where the temporal axis is treated analogously to spatial dimensions. This extension enables the algorithm to identify objects that maintain spatial and temporal coherence, linking detections across frames into continuous trajectories.

The ST-DBSCAN neighborhood definition incorporates an additional temporal epsilon parameter (ϵ_t) alongside the spatial epsilon. Points are considered neighbors if they satisfy both spatial proximity (within $\epsilon_{\text{spatial}}$ in x-y space) and temporal proximity (within $\epsilon_{\text{temporal}}$ frames). The geometric interpretation of this neighborhood varies by implementation: some formulations use a spherical neighborhood in 3D space-time, while others employ a cylindrical neighborhood where spatial and temporal extents are decoupled. The implemented version utilizes the cylindrical formulation, treating spatial and temporal scales independently to accommodate the different units and typical magnitudes of spatial versus temporal separations.

A key advantage of ST-DBSCAN for radar tracking is its robustness to temporary occlusions or detection failures. If a target disappears from radar returns for one or two frames, perhaps due to destructive interference, sea clutter, or aspect angle effects, but reappears in subsequent frames at a spatially consistent location, ST-DBSCAN can maintain track continuity by bridging the temporal gap. This capability exceeds that of frame-by-frame detection and matching approaches, which would terminate and reinitialize tracks across such gaps.

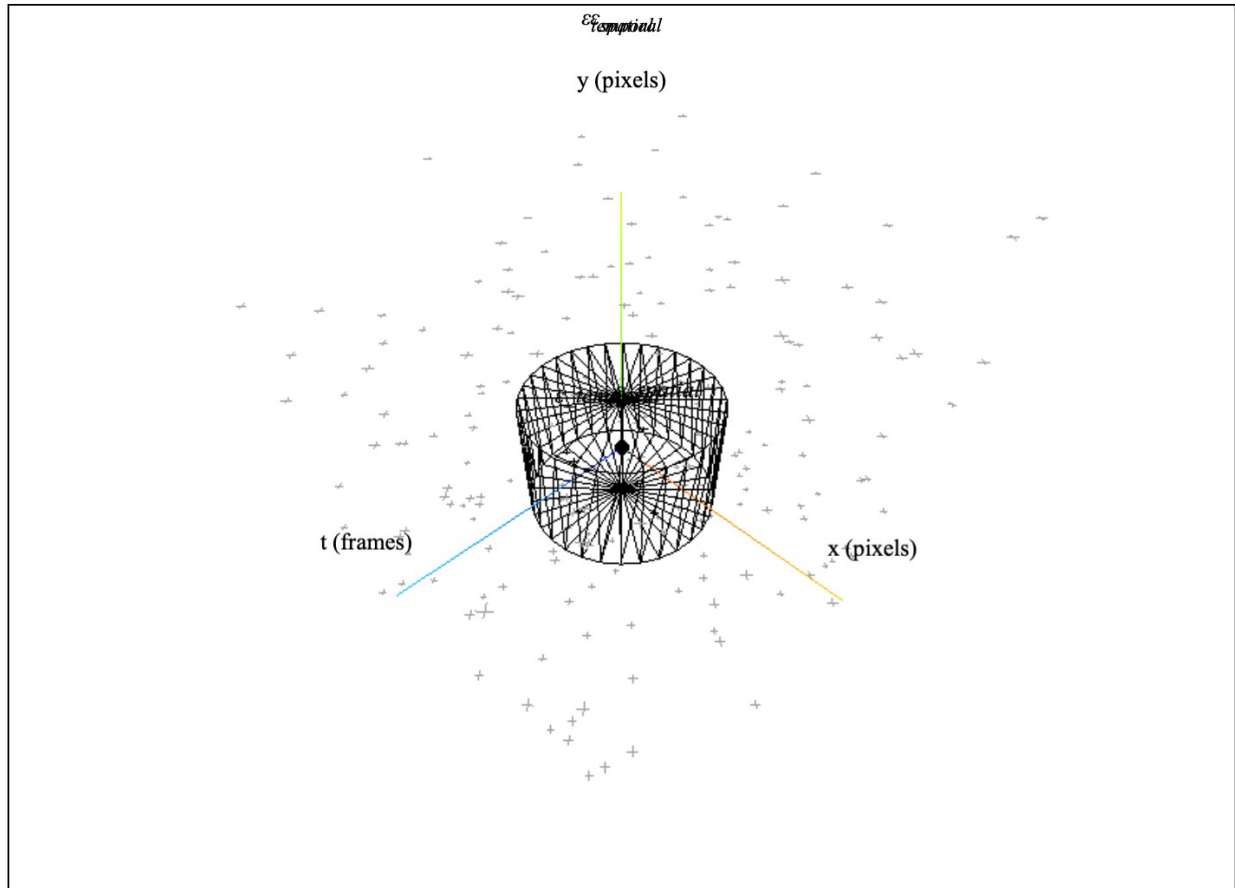


Figure 29: Visualization of the ST-DBSCAN methodology

However, the 3D nature of ST-DBSCAN introduces severe computational challenges. The algorithm must examine neighborhoods in a 3D space-time volume, substantially increasing the number of distance calculations required compared to 2D spatial clustering. Initial CPU-based implementations exhibited prohibitive execution times, taking minutes to process sequences of modest length. Parameter selection complexity also increases, as both spatial and temporal epsilon values require tuning, and their interaction effects on clustering behavior can be non-intuitive.

Part 8c: GPU Acceleration

To address the computational bottleneck of ST-DBSCAN, GPU acceleration was implemented for distance calculations and neighborhood queries. The inherently parallel structure of DBSCAN's neighborhood search operations maps naturally to GPU architectures. Each point's neighborhood can be evaluated independently, enabling massive parallelization across thousands of GPU cores.

The GPU implementation achieved dramatic speedups, reducing processing time from nearly 1,900 seconds per 20 frames to 500 on a GPU for 20 frames.

Part 8d: The Possibility of Point Cloud Representation

Future development will explore point cloud representations of radar data as an alternative to dense 2D image formats. Point clouds explicitly represent only the locations of radar returns above threshold, discarding empty regions and thereby reducing data dimensionality. Each point in the cloud corresponds to a radar return with associated spatial coordinates (x, y) and attributes such as intensity and possibly temporal information.

One promising approach involves using radar intensity as the third dimension of the point cloud, creating a 3D representation where x and y correspond to spatial position and z represents signal strength. This representation naturally emphasizes strong returns while de-emphasizing weak clutter, potentially improving the signal-to-noise characteristics for downstream processing. Alternatively, time can serve as the third dimension for temporal sequences, enabling 3D visualization and analysis of object trajectories through space-time.

Point cloud representations enable application of mature 3D computer vision algorithms developed for LiDAR and depth sensor data. Techniques such as PointNet and its successors for point cloud classification and segmentation, 3D object detection networks, and point cloud registration algorithms for tracking could potentially transfer to radar data with appropriate domain adaptation. This approach may offer advantages over treating radar as 2D images, as it naturally handles the sparse, non-uniform spatial distribution of radar returns.

Part 9: Data Collection – Nathan

High-quality ground truth data is essential for both training supervised learning models and validating algorithmic approaches. Data was captured at the Lake Murray dam 8 times, using the 2 provided data files. Implementation of Constant False Alarm Rate (CFAR) detection algorithms was developed this month, providing a classical baseline for target detection performance. An interactive parameter exploration tool enables rapid prototyping of CFAR configurations for different environmental conditions.

The screenshot shows a web interface titled "Maritime Radar CA-CFAR Processor". It features three sliders for parameter adjustment: "Guard Cells" set to 2, "Training Cells" set to 5, and "Probability of False Alarm (PFA)" set to 1e-4. Below the sliders, the algorithm is identified as "Cell Averaging Constant False Alarm Rate (CA-CFAR)" with a description: "CA-CFAR adaptively calculates detection thresholds based on local noise levels, reducing false alarms while preserving true targets." At the bottom, there are two buttons: "Select CSV Files" and "Generate & Process Images".

Maritime Radar CA-CFAR Processor

CA-CFAR Parameters

Guard Cells: 2 Training Cells: 5 Probability of False Alarm (PFA): 1e-4

Algorithm: Cell Averaging Constant False Alarm Rate (CA-CFAR)

Description: CA-CFAR adaptively calculates detection thresholds based on local noise levels, reducing false alarms while preserving true targets.

Select CSV Files Generate & Process Images

Figure 30: Adjustable parameter CFAR website on Replit

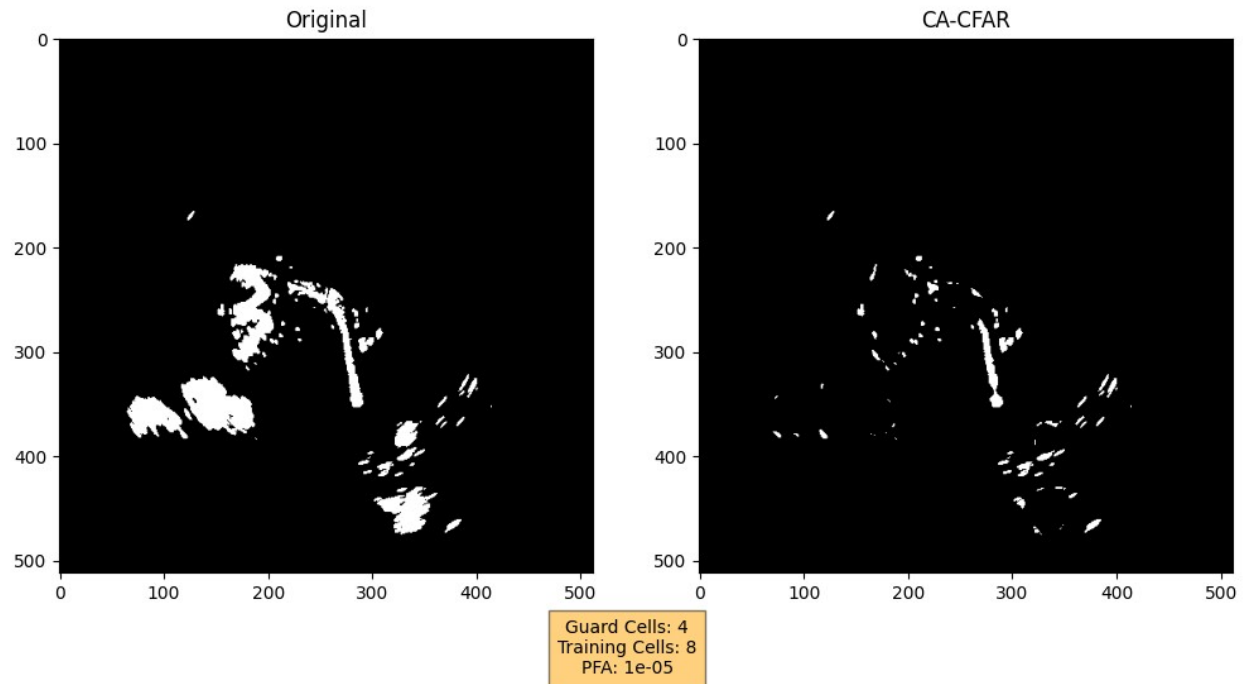


Figure 31: CFAR processed data

Part 10 Future

Part 10a: Algorithms and Model Development

Develop large MAE models; develop working variable-context temporal MAE models for velocity and direction predictions; continue work on data augmentation, CFAR, and DBSCAN work with undergraduates.

Part 10b: Experiments and Testing

Complete testing in Chesapeake Bay with second radar to add data diversity; continue collection in Lake Murray with standard settings; begin tests on high-resolution MAE on multi-GPU training to identify objects and detect small object locations.